Applicazione del RL e LLM al gioco Mr. Driller

Relazione di Progetto

Pesce Michele

10 aprile 2026

Sommario

Questo progetto esplora l'applicazione di tecniche di Reinforcement Learning (RL) e Large Language Models (LLM) per risolvere un livello del videogioco *Mr. Driller*. Vengono confrontati tre approcci: una baseline basata su Rainbow DQN puro, un agente guidato direttamente da un LLM per la scelta delle azioni, e un agente RL addestrato tramite un reward model basato su LLM. La valutazione su 10 episodi per agente mostra che l'approccio ibrido RL + LLM Reward ottiene il win rate più elevato (90%), superando sia il DQN puro (70%) sia l'LLM come action generator diretto (0%).

Indice

1 Introduzione **3**
1.1 Il Contesto di Gioco: Mr. Driller . . . 3
1.2 L'Ambiente di Simulazione . . . 3
1.3 Architettura dell'Agente RL e Training . . . 4
1.4 Integrazione e Valutazione dei Modelli LLM . . . 4
1.5 Obiettivi del Progetto . . . 4

2 Simulazione dell'Ambiente di Gioco **5**
2.1 Struttura della Griglia . . . 5
2.2 Generazione Procedurale dei Livelli . . . 5
2.3 Tipologie di Blocchi . . . 6
2.4 Connessione Visiva tra Blocchi: il Sistema di Tiling . . . 7
2.5 Il Personaggio . . . 8
2.6 Azioni Consentite . . . 8
2.7 Motore Fisico: Gravità dei Blocchi . . . 9
2.8 Coordinamento dell'Ambiente: il Loop Principale . . . 10

3 Sviluppo dell'Agente di Apprendimento per Rinforzo **11**
3.1 Rappresentazione dello Stato . . . 12
3.1.1 Tensore Visivo Locale . . . 12
3.1.2 Vettore di Stato Interno . . . 15
3.2 Architettura della Rete Neurale . . . 16

3.3 Algoritmo di Apprendimento: Rainbow DQN . . . 19
3.4 Ciclo Decisionale: ACTION–WAITING–LEARNING . . . 21
3.5 Funzione di Reward . . . 21
3.5.1 Componenti Terminali . . . 21
3.5.2 Componenti Non-Terminali . . . 22
3.6 Sistema Anti-Loop: DrillTracker . . . 23
3.7 Gestione dell'Esplorazione: Epsilon Adattivo . . . 24
3.8 Fase di Warmup e Curriculum Learning . . . 24
3.9 Persistenza e Monitoraggio del Training . . . 24

4 Integrazione dei Large Language Models 25
4.1 LLM come Action Generator . . . 26
4.2 LLM come Reward Model . . . 28
4.3 Framework di Benchmarking Comparativo . . . 29

5 Sperimentazioni, Sfide e Risultati 30
5.1 Agente Rainbow DQN: dal Prototipo alla Versione Finale . . . 30
5.1.1 Il problema della funzione di reward . . . 30
5.1.2 La fisica asincrona e il processo markoviano . . . 31
5.1.3 Sessione di Training e Dinamiche di Apprendimento . . . 31
5.2 LLM Agent e LLM Reward Model: Ottimizzazione e Validazione . . . 32
5.2.1 Il problema della latenza: dal prototipo all'agente ottimizzato . . . 32
5.2.2 Ingegnerizzazione del prompt e compromessi contestuali . . . 33
5.2.3 Risultati del Benchmark LLM . . . 33

6 Valutazione Comparativa dei Tre Agenti 34
6.1 Metodologia di Valutazione: il Modulo `evaluate_agents.py` . . . 34
6.2 Risultati Principali . . . 35
6.3 Cause di Fine Episodio . . . 36
6.4 Distribuzione delle Azioni . . . 37
6.5 Statistiche di Inferenza LLM durante la Valutazione . . . 37
6.6 Analisi Comparativa Complessiva . . . 38

7 Conclusioni 39

1 Introduzione

1.1 Il Contesto di Gioco: Mr. Driller

Figura 1: Esempio di un livello originale di Mr. Driller

Il videogioco *Mr. Driller* è un puzzle game dinamico e frenetico in cui il giocatore controlla un personaggio il cui scopo è trivellare verso il fondo di pozzi o livelli sotterranei. Le meccaniche principali ruotano attorno a una gestione costante e critica dell'ossigeno, che si esaurisce inesorabilmente con il passare del tempo e deve essere ripristinato raccogliendo apposite capsule disseminate lungo il percorso. Durante la discesa, il giocatore interagisce con diverse tipologie di blocchi: blocchi colorati standard, che scompaiono generando reazioni a catena se raggruppati in numero di quattro o più dello stesso colore, e i cosiddetti "blocchi X" (o blocchi marroni). Questi ultimi sono più resistenti e, se distrutti, penalizzano severamente il giocatore riducendo la sua riserva d'ossigeno. Un elemento cruciale del gameplay è la gravità: i blocchi privati di un supporto cadono verso il basso, creando il rischio costante di schiacciare il giocatore e fargli perdere una vita.

1.2 L'Ambiente di Simulazione

Per condurre gli esperimenti e addestrare gli agenti, è stato adottato e integrato un progetto open-source disponibile su GitHub, capace di simulare fedelmente dieci livelli del gioco originale. Sviluppato in Python tramite la libreria Pygame, questo ambiente non si limita a replicare la mera logica di gioco, ma offre un'esperienza visiva e sonora completa, provvista di menu interattivi ed effetti multimediali con sprite e figure creati ad hoc dagli sviluppatori.

1.3 Architettura dell'Agente RL e Training

Il cuore tecnologico del progetto risiede nello sviluppo di un agente di Reinforcement Learning (RL) basato sull'architettura avanzata Rainbow DQN. Questa soluzione integra e combina diverse estensioni del DQN classico. Nello specifico, sono stati implementati il *Double DQN*, che disaccoppia la selezione dell'azione dalla sua valutazione per mitigare la sovrastima dei valori Q, e il *Dueling DQN*, che divide la rete neurale in due flussi separati per stimare in modo indipendente il valore intrinseco di uno stato e il vantaggio fornito da ogni singola azione.

Per ottimizzare e stabilizzare la convergenza del modello, sono state adottate numerose tecniche all'avanguardia. È stata utilizzata la *Prioritized Experience Replay* (PER) per campionare con maggiore frequenza le transizioni più istruttive, affiancata da un *N-step buffer* per l'accumulo delle reward nel tempo (utilizzando un fattore di sconto $\gamma = 0.99$). L'esplorazione dello spazio delle azioni è stata garantita combinando un leggero decadimento del parametro Epsilon (*Epsilon Decay*) con l'introduzione di layer *Noisy*, i quali aggiungono casualità e rumore parametrico direttamente all'interno della rete. Per consentire all'agente di percepire la direzionalità e il movimento — come l'imminente caduta di un blocco — è stato implementato un *Frame Stacker*. Infine, il processo di training è stato perfezionato tramite uno scheduler del learning rate con andamento a coseno e una strategia di *Curriculum Learning*, addestrando l'agente dapprima su un singolo livello, poi su tre, fino a generalizzare le sue competenze su tutti e dieci i livelli, guidato da una funzione di reward appositamente disegnata per disincentivare lo scavo cieco ed esortare l'esplorazione strategica.

1.4 Integrazione e Valutazione dei Modelli LLM

Accanto all'agente RL puro, il progetto esplora l'integrazione dei Large Language Models (LLM) come strumenti di supporto alle decisioni e all'addestramento. Prima dell'implementazione, è stato condotto un benchmark su modelli gratuiti accessibili via API tramite OpenRouter, nello specifico `meta-llama/llama-3.1-8b-instruct` e `deepseek/deepseek-chat`, valutandone latenza ed efficacia in contesti di simulazione. Llama si è rivelato il modello più bilanciato e performante per i task richiesti, ed è stato pertanto selezionato per testare due varianti architetturali.

Dal confronto finale sui livelli simulati è emerso che l'agente con LLM come *Reward Model* ottiene le prestazioni migliori in assoluto (win rate 90%), superando il Rainbow DQN puro (70%). La variante che utilizza l'LLM come decisore diretto delle azioni (*Action Generator*) ha evidenziato un limite strutturale: pur essendo in grado di selezionare mosse valide, la necessità di processare prompt molto ampi per comprendere la griglia di gioco causa latenze troppo elevate. Questo trade-off rende, ad oggi, tale approccio inadatto per un task in tempo reale e altamente dinamico come *Mr. Driller*.

1.5 Obiettivi del Progetto

Alla luce di quanto esposto, gli obiettivi perseguiti da questo progetto possono essere così sintetizzati:

- Adozione e integrazione di un ambiente di simulazione realistico per il gioco *Mr. Driller*.

- Implementazione e addestramento scalabile di un agente RL puro basato su architettura Rainbow DQN.
- Sviluppo e valutazione comparativa di due varianti basate su LLM (Action Generator e Reward Model) per analizzarne l'efficacia e i limiti operativi in contesti decisionali in tempo reale.

2 Simulazione dell'Ambiente di Gioco

L'ambiente di gioco è stato implementato in Python utilizzando la libreria `pygame`, riproducendo fedelmente le meccaniche originali di *Mr. Driller*. Il punto di partenza è stato un clone open-source del gioco disponibile su GitHub [1], che forniva già la struttura di base dell'ambiente: la griglia, i blocchi, il personaggio e il loop di gioco. Su questa base sono stati integrati il motore fisico esteso, il sistema di ricompensa e tutti i componenti necessari all'addestramento dell'agente, descritti nei capitoli successivi.

L'ambiente si articola in tre strati principali: la griglia di gioco, il personaggio giocante e il motore fisico che ne governa l'interazione.

2.1 Struttura della Griglia

Il livello è rappresentato come una matrice bidimensionale $\mathcal{L} \in B^{R \times C}$, dove R è il numero di righe e $C = 7$ è la larghezza fissa della griglia. Ogni cella $\mathcal{L}[y][x]$ contiene un oggetto di tipo `Block`, caratterizzato da:

- **HP** (*hit points*): un intero ≥ 0; il valore 0 indica una cella vuota, cioè aria percorribile;
- **tipo**: una stringa che identifica la categoria del blocco (`"classic"`, `"unbreakable"`, `"delayed"`, `"solo"`, `"pill"`, `"end"`);
- **colore**: un intero in $\{1, \ldots, N_c\}$, rilevante solo per i blocchi `classic` ai fini della reazione a catena.

La struttura verticale di ogni livello si divide in tre zone:

1. **Buffer superiore** (righe 0–4): cinque righe di aria forzata che costituiscono la zona di spawn del personaggio.
2. **Zona di gioco** (righe 5–$R-1$): blocchi generati proceduralmente.
3. **Zona di uscita** (righe R e successive): blocchi di tipo `end` il cui contatto con il personaggio segnala il completamento del livello.

I livelli iniziali (1–4) hanno $R = 80$ righe, mentre quelli avanzati (5–10) salgono a $R = 155$ righe.

2.2 Generazione Procedurale dei Livelli

La generazione del livello avviene riga per riga, cella per cella, attraverso la funzione `generateLvl`. Per ogni cella della zona di gioco vengono estratti quattro numeri casuali indipendenti $u_i \sim \mathcal{U}(0, 100)$, uno per ciascuna categoria speciale di blocco. Il tipo assegnato alla cella segue una cascata di priorità a scalare: se la condizione per un tipo prioritario non è soddisfatta, si scende al successivo; in mancanza di qualsiasi corrispondenza, la cella riceve un blocco `classic` con colore estratto uniformemente.

Pill (priorità 1). Una capsula d'ossigeno viene piazzata se $u_{\text{pill}} < p_{\text{pill}}$, il budget totale $N_{\text{pill}} > 0$ e il contatore di distanza $d_{\text{pill}} < 0$. Il contatore parte da 0, viene decrementato di 1 a ogni nuova riga e reimposta a $d_{\text{min}} = 20$ ogni volta che una pill viene effettivamente piazzata. In questo modo le capsule non possono mai apparire a meno di 20 righe di distanza l'una dall'altra, evitando concentrazioni anomale.

Solo (priorità 2). Se $u_{\text{solo}} < p_{\text{solo}}$, la cella riceve un blocco `solo`: si distrugge con un colpo ma non innesca reazioni a catena.

Unbreakable (priorità 3). Se $u_{\text{unb}} < p_{\text{unb}}$, viene inserito un blocco `unbreakable` con 5 HP, pensato per ostacolare il progresso del giocatore e penalizzarne l'ossigeno.

Delayed (priorità 4). Se $u_{\text{del}} < p_{\text{del}}$, la cella riceve un blocco `delayed`: non si rompe istantaneamente ma avvia un countdown di 2 secondi prima di scomparire.

Classic (default). In tutti gli altri casi la cella riceve un blocco `classic` con colore estratto uniformemente in $\{1, \ldots, N_c\}$. Questo è il tipo più comune e l'unico a partecipare alla reazione a catena.

I valori di default dei parametri di generazione sono riportati in Tabella 1. Per i livelli avanzati (5–10) il parametro p_{del} viene portato a 12% e p_{unb} a 10%, rendendo il livello progressivamente più impegnativo.

Tabella 1: Parametri di generazione procedurale del livello.

Parametro	**Simbolo**	**Valore default**
Probabilità Pill	p_{pill}	5%
Budget massimo Pill	N_{pill}	7
Distanza minima tra Pill	d_{min}	20 righe
Probabilità Solo	p_{solo}	10%
Probabilità Unbreakable	p_{unb}	5%
Probabilità Delayed	p_{del}	10%

2.3 Tipologie di Blocchi

Classic.
Blocco colorato standard con HP = 1. Quando viene distrutto, la reazione a catena si propaga ricorsivamente ai quattro vicini ortogonali che condividono lo stesso colore, potenzialmente eliminando interi cluster di blocchi con un singolo colpo.

Unbreakable.
Blocco resistente con HP = 5, privo di reazione a catena. Ogni colpo ne riduce gli HP di 1 e aggiorna visivamente la texture per mostrare il progressivo deterioramento. La sua distruzione penalizza il personaggio con −20% di ossigeno.

Delayed.
Blocco cristallo con HP effettivi gestiti internamente. Al primo colpo avvia un countdown di 2 secondi, scandito dall'evento `EVTICSEC`, al termine del quale scompare automaticamente. Il meccanismo è implementato nel metodo `timeout`: ad ogni tick il contatore interno `__seconds` viene decrementato; quando raggiunge 0 l'HP scende a 0 e il blocco viene rimosso dalla griglia.

Solo.
Blocco con HP = 1 e `chain_reaction = 0`: si distrugge con un singolo colpo senza alcun effetto sui vicini.

Pill.
Capsula d'ossigeno con HP = 1. Non può essere perforata con il trapano; si raccoglie quando il personaggio ci cammina sopra o la scala. La raccolta aggiunge +20% di ossigeno oppure riporta al 100% se l'ossigeno è già superiore al 70%.

End.
Blocco di fine livello. Il contatto del personaggio con questa cella genera un evento `pygame.USEREVENT` che segnala al loop principale il completamento del livello.

2.4 Connessione Visiva tra Blocchi: il Sistema di Tiling

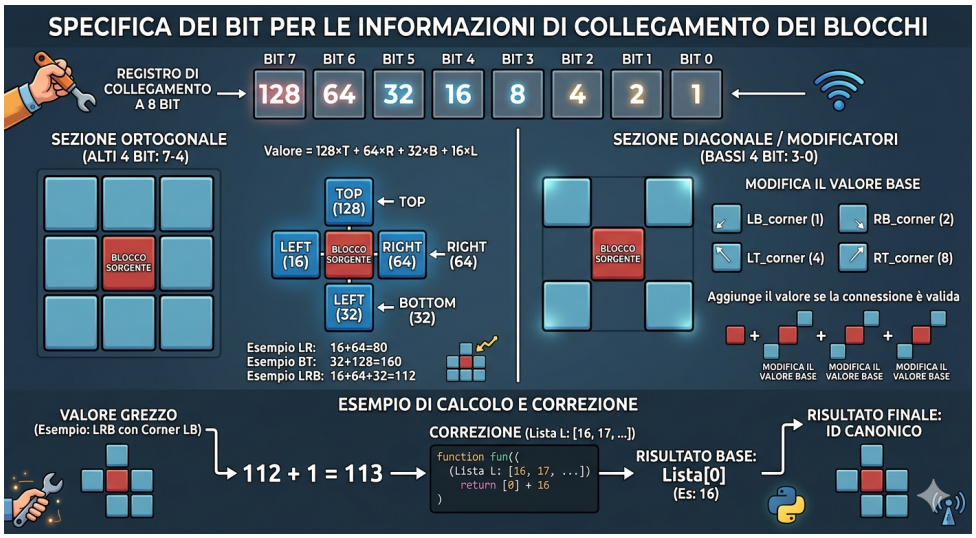


Figura 2: Rappresentazione visiva sul funzionamento del sistema di tiling

Uno degli aspetti più caratteristici dell'estetica di *Mr. Driller* è che i blocchi dello stesso colore adiacenti si fondono visivamente, formando strutture continue invece di apparire come elementi separati. Questo effetto è realizzato tramite un sistema di *tiling* basato su una maschera a 8 bit, implementato nel metodo `updCoText` della classe `Block` e nella funzione `correct` del modulo `connectCorrect`.

Per ogni blocco `classic`, il metodo `updCoText` analizza i suoi otto vicini (quattro ortogonali e quattro diagonali). Per ciascuna delle quattro direzioni ortogonali, un bit

viene acceso se il vicino è dello stesso colore, ha HP > 0 ed è anch'esso di tipo `classic`. I valori dei bit sono:

Direzione	**Contributo alla maschera**
Sinistra	+16
Basso	+32
Destra	+64
Alto	+128

I bit diagonali (+1, +2, +4, +8) vengono accesi solo se *entrambi* i lati ortogonali adiacenti a quella diagonale sono già connessi: per esempio, l'angolo in basso a sinistra (+1) è attivo solo se sono attivi sia il bit del basso che quello di sinistra. Questo evita connessioni angolari spurie.

Il valore numerico risultante $v \in [0, 255]$ serve come indice per selezionare la texture grafica corrispondente. La funzione `correct` del modulo `connectCorrect` si occupa di normalizzare i valori non validi, mappandoli al rappresentante canonico del gruppo di appartenenza. Se il valore normalizzato corrisponde a un file `.png` esistente nella cartella del colore, quella texture viene caricata; altrimenti si tenta la versione senza angoli (v & 240), e in ultima istanza si ricade su `0.png`, la texture di un blocco completamente isolato.

Questo sistema viene ricalcolato sull'intera griglia ogni volta che la fisica modifica la disposizione dei blocchi, garantendo sempre una rappresentazione visiva coerente.

2.5 Il Personaggio

Il personaggio è descritto da una posizione discreta (x, y) nella griglia, un livello di ossigeno $o \in [0, 100]$, un punteggio $s \geq 0$ e un numero di vite $l \geq 0$. Le coordinate di spawn sono fisse: $x_0 = 3$, $y_0 = 4$.
L'ossigeno decresce di 1% a ogni tick di `EVTICSEC`. Quando $o \leq 0$, il personaggio perde una vita e torna alla posizione iniziale con $o = 100\%$; se $l < 0$ la partita termina.
La **telecamera** è gestita da un offset $\delta \in N$ che conta quante righe sono uscite dalla sommità dello schermo. In ogni istante vengono renderizzate le righe $[\delta, \delta + 8]$. Ogni volta che il personaggio scende di una riga, δ viene incrementato e propagato a tutti i blocchi tramite `updOffset`.

2.6 Azioni Consentite

Ad ogni passo di decisione il personaggio può eseguire una delle sei azioni discrete elencate in Tabella 2. La gestione dell'input è delegata al modulo `eventHandling`, che espone due funzioni: `movementHandle`, per i movimenti laterali, e `breaking`, per i colpi di trapano. Entrambe ricevono un evento `pygame` e traducono il tasto premuto nella chiamata ai metodi corrispondenti del personaggio.

Tabella 2: Spazio delle azioni dell'agente.

Indice	Nome	Descrizione
0	IDLE	Nessuna operazione; il personaggio rimane fermo.
1	LEFT	Movimento a sinistra verso $\mathcal{L}[y][x-1]$, se la cella è vuota o contiene una pill.
2	RIGHT	Movimento a destra verso $\mathcal{L}[y][x+1]$, simmetrico a LEFT.
3	DRILL_LEFT	Colpo di trapano su $\mathcal{L}[y][x-1]$, se HP > 0 e il blocco non è una pill.
4	DRILL_RIGHT	Colpo di trapano su $\mathcal{L}[y][x+1]$, simmetrico a DRILL_LEFT.
5	DRILL_DOWN	Colpo di trapano su $\mathcal{L}[y+1][x]$, se HP > 0, il blocco non è una pill e $y < \delta + 8$.

Il movimento laterale include una meccanica di **scalata automatica**: se la cella frontale è occupata ma quella diagonalmente superiore è libera, il personaggio si sposta di $(\pm 1, -1)$ incrementando un contatore interno `climb`. La gravità riporta poi il personaggio alla quota corretta decrementando tale contatore di uno per ogni tick fisico.

2.7 Motore Fisico: Gravità dei Blocchi

Il motore fisico è attivato dall'evento periodico `EVTIC_GRAVITY` e implementato nella funzione `applyGravity`. La scansione avviene **dal basso verso l'alto** (da riga $R-2$ a riga 0): in questo modo ogni blocco può scendere al massimo di una cella per tick, impedendo propagazioni istantanee verso il fondo. Solo i blocchi di tipo `classic`, `solo` e `delayed` con HP > 0 sono soggetti alla gravità.

Fase A — Rilevamento del vuoto e incollamento. Se la cella sottostante è aria (HP $= 0$), si controlla se il blocco è *incollato* a un vicino. Un blocco `classic` è incollato se almeno uno dei vicini ortogonali (sinistra, destra, o sopra) condivide il suo colore e ha HP > 0. In questo caso il blocco rimane fermo, altrimenti viene avviato il tremore.

Fase B — Tremore pre-caduta. Il blocco entra nello stato `isShaking` e ad ogni tick viene incrementato un contatore interno. Dopo `SHAKE_TICKS_BEFORE_FALL` $= 5$ tick il blocco transisce allo stato `isFalling`. Questa fase dà al giocatore un preavviso visivo prima che il blocco cada effettivamente.

Fase C — Caduta libera. Il blocco viene spostato nella cella sottostante tramite scambio diretto: $\mathcal{L}[y+1][x] \leftrightarrow \mathcal{L}[y][x]$, con aggiornamento delle coordinate interne. Se la cella di destinazione coincide con la posizione del personaggio, questi perde una vita. Il blocco torna a riposo (`stopFalling`) non appena la cella sottostante ha HP > 0.

Anche la gravità agisce sul **personaggio**: se la cella sotto è vuota o contiene una pill, il personaggio scende di una riga. Per evitare interazioni fisiche anomale, il personaggio *non* scende se il blocco sottostante è nello stato `isFalling` o `isShaking`: in questo caso la discesa viene posticipata al tick successivo.

Al termine di ogni ciclo di gravità, se almeno un blocco ha cambiato posizione, le texture di connessione di tutti i blocchi vengono ricalcolate tramite il sistema di tiling descritto nella Sezione 2.4.

2.8 Coordinamento dell'Ambiente: il Loop Principale

Tutta la logica dell'ambiente è coordinata dalla funzione `game` in `main.py`, che implementa un loop infinito fondato sul sistema di eventi di `pygame`. Ogni iterazione del loop gestisce tre macro-aree: la ricezione degli eventi, l'aggiornamento dello stato e il rendering visivo.

Sistema a eventi. `pygame` mette a disposizione una coda di eventi alla quale sia il sistema operativo (input da tastiera, chiusura della finestra) sia il codice stesso possono accodare messaggi. Il gioco sfrutta questa coda definendo quattro *timer personalizzati*, ciascuno associato a un identificatore `USEREVENT + k`, che scattano a intervalli regolari e guidano la dinamica dell'ambiente (Tabella 3).

Tabella 3: Timer personalizzati del loop di gioco.

Evento	**Funzione**	**Periodo (ms)**
`EVTIC_GRAVITY`	Gravità blocchi e personaggio	400/`GS`
`EVTIC_ANIM`	Aggiornamento sprite personaggio	80/`GS`
`EVTICSEC`	Consumo ossigeno, timeout Delayed	1050/`GS`
`EVTICPLY`	Reset stato idle personaggio	120/`GS`

dove `GS` = `GAME_SPEED` è il moltiplicatore globale di velocità (default 1.0).

Modalità di esecuzione. Il loop supporta due modalità operative. In **modalità display** (`HEADLESS_TRAINING = False`) i timer reali di `pygame` scandiscono gli eventi e il rendering è eseguito a 60 FPS: questa modalità è utilizzata per osservare l'agente in azione. In **modalità headless** (`HEADLESS_TRAINING = True`) i driver video e audio vengono sostituiti da driver fittizi (`dummy`), il rendering è disabilitato e gli eventi vengono iniettati direttamente nella coda a ogni iterazione del loop, massimizzando il numero di step per secondo durante l'addestramento.

Gestione della progressione. Il completamento di un livello è segnalato dall'evento `EVT_END_BLOCK`, generato dal blocco `End` al momento del contatto con il personaggio. A questo punto il loop chiama `changeLvl`, che genera un nuovo livello con i parametri aggiornati e riposiziona il personaggio. Se il livello completato è l'ultimo disponibile (> 10), viene invece invocata `storeScore` per salvare il punteggio e la partita termina. In caso di *game over* (vite esaurite), la funzione `restart` riporta l'ambiente allo stato iniziale, rigenerando il livello 1 e azzerando punteggio e vite, pronto per un nuovo episodio.

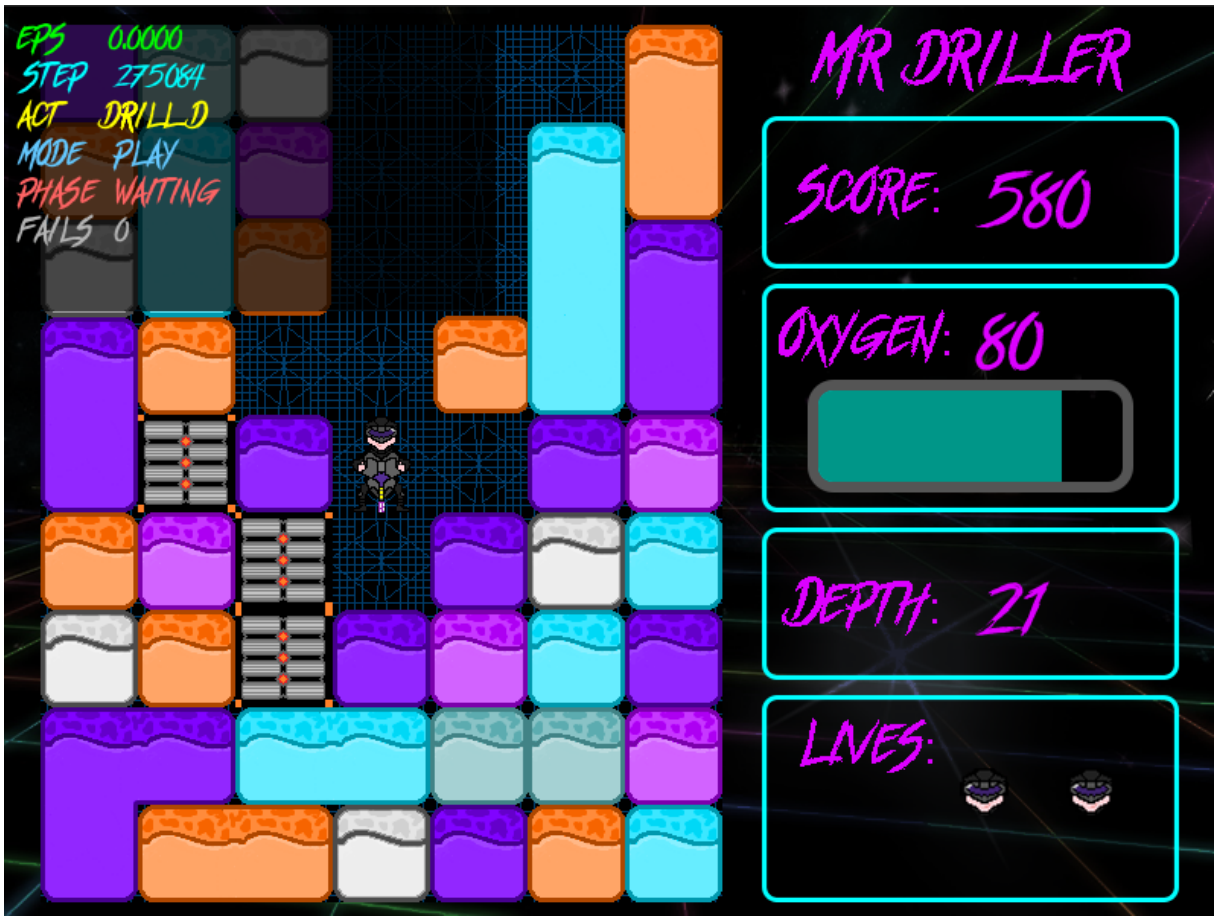


Figura 3: Esempio del livello 1 dell'ambiente simulato

3 Sviluppo dell'Agente di Apprendimento per Rinforzo

Il presente capitolo descrive in dettaglio l'architettura e i meccanismi dell'agente sviluppato per affrontare il problema del controllo autonomo in *Mr. Driller*. L'approccio adottato è quello del *Deep Reinforcement Learning* (Deep RL), una famiglia di metodi che combinano l'apprendimento per rinforzo classico con reti neurali profonde in grado di approssimare funzioni di valore su spazi di stato ad alta dimensionalità.

L'apprendimento per rinforzo è formalmente inquadrabile nel paradigma dei *Markov Decision Process* (MDP), definito dalla tupla $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$, in cui $\mathcal{S}$ è lo spazio degli stati, $\mathcal{A}$ lo spazio delle azioni, $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ la funzione di transizione, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to R$ la funzione di reward e $\gamma \in [0, 1)$ il fattore di sconto. L'obiettivo dell'agente è trovare una politica $\pi^* : \mathcal{S} \to \mathcal{A}$ che massimizzi il ritorno atteso:

$$G_t = E\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k}\right] \tag{1}$$

L'agente implementa una variante della famiglia *Rainbow DQN* [2], integrando selettivamente le estensioni più rilevanti per le caratteristiche del dominio: *Dueling Networks*, *Noisy Nets*, *Prioritized Experience Replay* (PER) e *N-step Returns*. La scelta di non adottare l'intero Rainbow (che include anche *Distributional RL* e *C51*) è motivata dall'analisi del

dominio: la distribuzione della reward in *Mr. Driller* non presenta le multimodalità marcate che rendono il Distributional RL particolarmente vantaggioso [3], mentre le quattro estensioni selezionate affrontano direttamente i problemi identificati nella fase esplorativa: instabilità del training, esplorazione insufficiente, lento propagarsi del segnale di reward e sovrastima sistematica dei Q-values. Questi problemi sono stati osservati empiricamente durante le prime sessioni di addestramento su una policy DQN baseline e hanno guidato la progressiva adozione di ciascuna estensione, valutandone il contributo incrementale.

3.1 Rappresentazione dello Stato

Una delle sfide centrali nell'applicazione del Deep RL a un gioco platform è la definizione di una rappresentazione dello stato $s_t \in \mathcal{S}$ che sia simultaneamente *markoviana* (contenga tutta l'informazione necessaria per la decisione ottimale), *compatta* (non introduca ridondanza computazionale inutile) e *numericamente stabile* (si presti al processing da reti neurali profonde senza richiedere normalizzazioni complesse).

Un primo approccio naturale, ispirato al DQN originale di Mnih et al. [4], consiste nell'usare come stato direttamente i pixel grezzi dello schermo. Questa scelta è stata deliberatamente scartata per due ragioni principali: in primo luogo, la risoluzione grafica di *Mr. Driller* introduce rumore visivo (animazioni, sfondi, effetti) del tutto irrilevante per la strategia di gioco; in secondo luogo, la rete dovrebbe apprendere *da zero* concetti come "questo blocco è indistruttibile" o "questo personaggio sta cadendo", rallentando enormemente la convergenza. Un'ingegnerizzazione esplicita dello stato consente invece di fornire all'agente una rappresentazione già semanticamente ricca, riducendo il carico computazionale e accelerando l'apprendimento.

In questo lavoro lo stato s_t è composto da due componenti complementari progettate per catturare rispettivamente la struttura spaziale dell'ambiente e le variabili di processo interne al personaggio: un tensore visivo locale e un vettore scalare.

3.1.1 Tensore Visivo Locale

Trasmettere all'agente l'intera griglia di gioco, che può raggiungere le 155×7 celle, sarebbe computazionalmente oneroso e introdurrebbe ridondanza informativa: le regioni lontane dal giocatore non possono essere raggiunte in pochi passi e non contribuiscono alla decisione ottimale nell'immediato. È pertanto stato scelto di estrarre una finestra locale centrata sul personaggio, di dimensioni 11×9 celle (11 righe, 9 colonne), implementata nella funzione `get_local_window_tensor`. Le dimensioni sono state scelte in modo da garantire visibilità sufficiente nelle direzioni critiche: 5 righe sopra la testa (per rilevare blocchi in caduta) e 5 righe sotto i piedi (per pianificare il percorso discendente), con 4 colonne di contesto laterale per ciascun lato. Le coordinate assolute nella mappa corrispondenti alla posizione (i, j) nella finestra sono:

$$\text{abs_y}(i) = p_y - d_y + i, \qquad \text{abs_x}(j) = p_x - d_x + j \tag{2}$$

Quando queste coordinate escono dai limiti della mappa, le celle corrispondenti vengono trattate come muri esterni: il canale 0 riceve un valore strategico basso (0.1) e il canale 2 riceve un valore di durezza massima (1.0). Questa scelta garantisce che i bordi della mappa vengano percepiti dalla rete esattamente come blocchi indistruttibili, evitando comportamenti patologici come il tentativo di uscire dai confini del livello.

La finestra è codificata in **6 canali semantici distinti**, ciascuno dei quali cattura una dimensione diversa dell'informazione ambientale. La scelta di separare l'informazione in canali tematici distinti, anziché usare un singolo canale numerico, è motivata dalla necessità di non creare ambiguità: un blocco con HP=1 e un timer attivo ha un significato completamente diverso da un blocco con HP=1 che è semplicemente usurato. La separazione semantica permette alla rete convoluzionale di apprendere filtri specializzati per ciascuna dimensione informativa.

Canale 0 — Valore strategico. Ad ogni cella viene assegnato un punteggio in $[0, 1]$ che riflette l'utilità strategica del blocco presente, secondo la tabella di mapping `_STRATEGIC_VALUES`. Questo canale serve a guidare l'attenzione della rete verso le celle più rilevanti per il progresso della partita: il blocco `end` è l'obiettivo finale del livello, le `pill` di ossigeno sono risorse critiche per la sopravvivenza, mentre i muri indistruttibili non apportano nessun beneficio e vanno evitati. Il fatto che i valori siano continui e ordinati (da 0.1 a 1.0) consente ai filtri convoluzionali di apprendere risposte graduate alla "priorità" di ciascun blocco, piuttosto che doverla dedurre implicitamente da un identificatore categorico.

Tabella 4: Valori strategici assegnati ai diversi tipi di blocco nel Canale 0.

Tipo blocco	Valore strategico	Motivazione
`end`	1.0	Blocco-obiettivo: porta alla vittoria del livello
`pill`	0.9	Capsula ossigeno: risorsa critica
`classic`	0.7	Blocco normale: perforabile e fonte di punteggio
`delayed`	0.6	Blocco a tempo: richiede pianificazione
`solo`	0.5	Blocco singolo: neutro, non si attacca
`unbreakable`	0.1	Muro indistruttibile: da evitare

Canale 1 — Colore e potenziale di combo. Per i soli blocchi `classic`, il canale codifica l'indice del colore normalizzato in $[0, 1]$, sommato a un *bonus combo* pari a 0.25 per ciascun vicino ortogonale dello stesso colore con HP > 0. La meccanica delle combo è uno degli elementi più complessi di *Mr. Driller*: quando due o più blocchi dello stesso colore si toccano, perforandone uno si distrugge l'intera catena, ottenendo molti più punti. Questo bonus rende la rete consapevole della potenziale estensione della reazione a catena, incoraggiandola implicitamente a preferire perforazioni che attivano combo di grandi dimensioni.

Canale 2 — Durezza e timer residuo. Il canale codifica la "resistenza" della cella in termini fisici. La codifica differenziata per tipo di blocco è necessaria perché l'informazione rilevante è qualitativamente diversa da tipo a tipo: blocchi `delayed` con countdown attivo ricevono valore = `timer`/2, permettendo alla rete di percepire quanto manca prima che il blocco si attivi e divenga pericoloso; blocchi `unbreakable` ricevono = HP/5, codificando il fatto che ogni tentativo di perforazione li danneggia ma non li distrugge mai; tutti gli altri blocchi ricevono = HP, ovvero quanti colpi ancora servono per eliminarli.

Canale 3 — Urgenza fisica e traiettoria di caduta. Questo è il canale più critico per la sopravvivenza dell'agente. Quando un blocco viene perforato, i blocchi soprastanti che non hanno più supporto iniziano a tremare e poi cadono. Un agente che non percepisce

questa minaccia muore sistematicamente. Per ogni cella, si calcola un valore di urgenza $u \in [0, 1]$:

$$u_{i,j} = \begin{cases} 0.95 \cdot w_{\text{col}} & \text{se il blocco cade e } r_y \leq 0 \\ 0.50 \cdot w_{\text{col}} & \text{se il blocco trema e } r_y < 0 \\ 0 & \text{altrimenti} \end{cases} \tag{3}$$

dove $r_y = i - \lfloor h/2 \rfloor$ è la posizione verticale relativa al centro della finestra e il peso colonna:

$$w_{\text{col}} = \frac{1}{1 + |j - j_{\text{center}}| \cdot 0.7} \tag{4}$$

Il peso di colonna w_{col} riflette il fatto che un blocco in caduta direttamente sopra la testa ($j = j_{\text{center}}$) è letale con certezza, mentre uno in caduta a tre colonne di distanza può essere evitato spostandosi di poco. La distinzione tra caduta (0.95) e tremore (0.50) codifica una gradazione temporale: il tremore è un segnale di allerta precoce, la caduta indica un pericolo immediato.

Una seconda scansione *propaga la traiettoria di caduta*: se una cella ha urgenza ≥ 0.80 e le celle sottostanti sono aria vuota, esse vengono riempite con un'urgenza decrescente:

$$u_{\text{traj}}(k) = u_{\text{sorgente}} \cdot 0.85^k \tag{5}$$

dove k è la distanza in celle dalla sorgente. Questa propagazione è essenziale perché la rete possa "vedere" dove atterrerà il blocco in caduta, non solo da dove parte.

Canale 4 — Peso di prossimità. Un gradiente spaziale fisso che assegna a ogni cella un peso inversamente proporzionale alla sua distanza dal centro della finestra:

$$w_{i,j} = \frac{1}{1 + \sqrt{(r_y \cdot 0.8)^2 + (|r_x| \cdot 2.5)^2}} \tag{6}$$

Il peso orizzontale (2.5) è maggiore di quello verticale (0.8) per riflettere l'asimmetria del dominio: il giocatore si muove principalmente in verticale, per cui le celle laterali sono meno rilevanti di quelle immediatamente sopra e sotto. Questo canale non porta informazione ambientale aggiuntiva, ma funge da *bias induttivo spaziale*: poiché i filtri convoluzionali non hanno di per sé un senso di "dove si trova il centro", questo gradiente fornisce loro un ancoraggio spaziale esplicito.

Canale 5 — Mappa del vuoto. Un canale binario che vale 1.0 se la cella è aria (HP = 0) e 0.0 altrimenti. Sebbene il vuoto possa sembrare già implicito dalla combinazione degli altri canali, renderlo esplicito permette ai filtri di apprendere direttamente le nozioni di "spazio percorribile" e "percorso libero verso il basso", fondamentali per la navigazione.

Il tensore finale ha dimensioni $(6, 11, 9)$. Per dare alla rete una percezione del *movimento* nel tempo, che non è catturabile da un singolo frame (un blocco che sta cadendo ha lo stesso valore di uno fermo, ma la sua traiettoria è deducibile solo dalla variazione temporale), i frame vengono accumulati in uno **stack temporale** di $N_{\text{frames}} = 4$ frame consecutivi tramite la classe `SmartFrameStacker`. Quattro frame corrispondono a quattro decisioni consecutive dell'agente, un orizzonte sufficiente a distinguere un blocco in movimento da uno statico e a percepire la velocità di caduta. Il tensore di input effettivo alla rete neurale ha quindi dimensioni:

$$(N_{\text{channels}} \times N_{\text{frames}},\ 11,\ 9) = (6 \times 4,\ 11,\ 9) = (24,\ 11,\ 9) \tag{7}$$

3.1.2 Vettore di Stato Interno

Il tensore visivo descrive l'ambiente locale, ma non è sufficiente da solo. Esistono informazioni di processo che non emergono dall'osservazione diretta della griglia: quanta ossigeno manca al personaggio, quante vite ha a disposizione, in che direzione sta guardando, se si trova in uno stato fisico di caduta. Alcune di queste informazioni sarebbero tecnicamente deducibili dalla sequenza di frame (la direzione del personaggio cambia il suo sprite), ma richiederebbero che la rete impari a estrarre questa informazione in modo implicito, con un costo computazionale e di convergenza non giustificato. È quindi stato progettato un vettore scalare $\mathbf{v} \in R^{32}$, costruito dalla funzione `get_internal_state_vector`, che fornisce direttamente all'agente queste informazioni privilegiate.

1. **Slot 1–6**: posizione normalizzata $(x/C, y/R)$, ossigeno $(o/100)$, vite residue, punteggio e profondità. La normalizzazione in $[0, 1]$ è essenziale per evitare che i gradienti nelle fasi iniziali del training siano dominati da feature con range molto diversi (es. punteggio vs. posizione).

2. **Slot 7–8**: flag binari per soglie critiche di ossigeno ($o < 20$, stato di emergenza; $o < 50$, stato di allerta). Sebbene il livello di ossigeno sia già presente allo Slot 3, l'esperienza empirica ha mostrato che senza questi flag la rete faticava a discriminare la fase critica: aggiungere un segnale discontinuo e forte (0 o 1) per le soglie di pericolo funge da "campanello d'allarme" che il modello lineare dei layer densi riesce ad associare direttamente con comportamenti specifici (es. avvicinarsi urgentemente a una capsula).

3. **Slot 9–12**: direzione corrente in codifica one-hot. La codifica one-hot è preferita a un singolo intero perché i layer lineari trattano le feature come numeri continui: codificare "sinistra=1, destra=2" suggerirebbe implicitamente che destra è "il doppio" di sinistra, una relazione ordinale priva di senso semantico.

4. **Slot 13–18**: ultima azione eseguita in codifica one-hot su 6 classi. Fornire l'azione precedente riduce il problema di parziale osservabilità: consente all'agente di distinguere stati che appaiono identici visivamente ma hanno origini diverse, utile per ad esempio evitare ripetizioni inutili di un'azione fallita.

5. **Slot 19–22**: HP dei quattro vicini ortogonali, normalizzati a $[0, 1]$. Questi valori, già parzialmente presenti nel Canale 2 del tensore visivo, vengono qui replicati per renderli immediatamente accessibili alle teste di valore e vantaggio, riducendo il numero di trasformazioni necessarie per estrarre questa informazione critica dal branch convoluzionale.

6. **Slot 23–26**: flag binari per la presenza di pill nelle quattro direzioni adiacenti. Analogamente ai flag di ossigeno critico, questi indicatori discreti facilitano l'apprendimento di una risposta immediata alla disponibilità di risorse vitali nelle vicinanze.

7. **Slot 27–32**: indicatori fisici avanzati: flag di caduta (il personaggio è attualmente in aria?), timer d'aria (quanti frame ha trascorso in aria), instabilità del blocco sottostante (il pavimento sta per cedere?), distanza dal blocco in caduta sovrastante, contatore pericoli verticali nella colonna corrente, pericolo nelle colonne adiacenti. Questi ultimi slot nascono dall'osservazione che molte morti dell'agente nelle fasi

iniziali erano causate da situazioni fisiche complesse che non erano sufficientemente salienti nel tensore visivo: aggiungere il computo esplicito di queste grandezze ha ridotto significativamente la mortalità da caduta.

3.2 Architettura della Rete Neurale

L'architettura della rete, implementata nella classe `DrillerDuelingDQN`, è progettata per processare le due componenti dello stato in parallelo tramite due rami specializzati, fonderle in un embedding condiviso, e produrre una stima dei valori Q per ciascuna delle 6 azioni disponibili. Un diagramma completo dell'architettura è riportato in Figura 4.

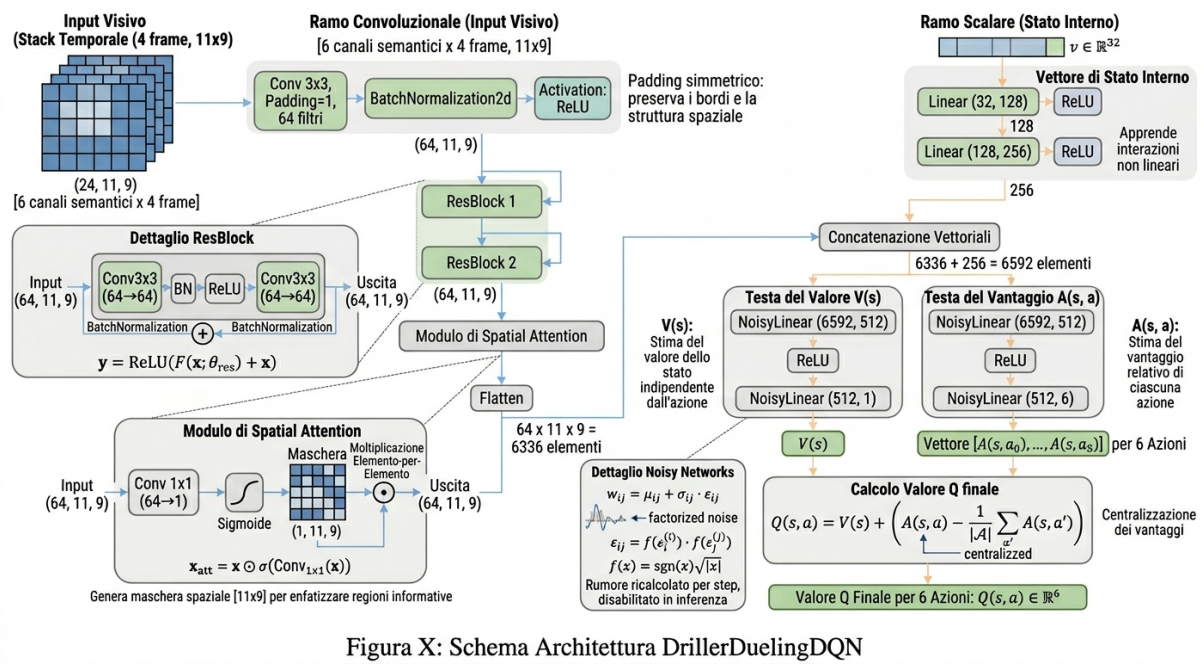


Figura X: Schema Architettura DrillerDuelingDQN

Figura 4: Schema completo dell'architettura `DrillerDuelingDQN`: il ramo convoluzionale (sinistra) processa il tensore visivo $(24, 11, 9)$ attraverso una sequenza di layer convoluzionali, ResBlock e un modulo di Spatial Attention; il ramo scalare (destra) processa il vettore interno $\mathbf{v} \in R^{32}$ tramite due layer lineari; i due embedding vengono fusi e processati dalla testa Dueling per produrre il valore Q finale $Q(s, a) \in R^6$.

Ramo convoluzionale (input visivo). Il tensore $(24, 11, 9)$ viene processato da una sequenza di moduli. La scelta di reti convoluzionali per processare il tensore visivo è naturale: le convoluzioni sfruttano la struttura spaziale locale dei dati, condividendo i pesi tra tutte le posizioni della griglia. In altri termini, un filtro che impara a riconoscere "blocco in caduta sopra al giocatore" sarà ugualmente utile indipendentemente da dove si trovino nella finestra, grazie alla proprietà di *equivarianza traslazionale*.

1. Un layer convoluzionale con 64 filtri di kernel 3×3 e padding $= 1$, seguito da `BatchNormalization2d` e attivazione `ReLU`. Il padding simmetrico preserva le dimensioni spaziali 11×9, evitando che le informazioni ai bordi della finestra vengano perse nelle fasi iniziali dell'elaborazione. La `BatchNormalization` normalizza le attivazioni dopo ogni strato, stabilizzando il training e permettendo l'uso di learning rate più elevati senza rischio di esplosione dei gradienti.

2. **Due blocchi residuali** (`ResBlock`): il problema dell'addestramento di reti profonde è che i gradienti tendono a "svanire" durante la backpropagation, rendendo i layer più profondi difficili da aggiornare. I ResBlock introducono una connessione skip additiva che permette al gradiente di fluire direttamente dall'output all'input del blocco, aggirando la non-linearità:

$$\mathbf{y} = \mathrm{ReLU}(\mathcal{F}(\mathbf{x}; \theta_{\mathrm{res}}) + \mathbf{x}) \tag{8}$$

   Ogni blocco è composto da due layer convoluzionali 3×3 con normalizzazione. La rete impara quindi la *funzione residuale* $\mathcal{F}(\mathbf{x})$ piuttosto che la trasformazione completa $\mathbf{y}(\mathbf{x})$: se la miglior trasformazione è l'identità (cioè il layer non aggiunge informazione utile), la rete può imparare semplicemente $\mathcal{F}(\mathbf{x}) \approx 0$, operazione molto più facile per un ottimizzatore a gradiente.

3. **Modulo di Attenzione Spaziale** (`SpatialAttention`): non tutte le celle della finestra sono ugualmente rilevanti per la decisione corrente. Se l'ossigeno è al 15%, la capsula più vicina è l'informazione dominante; se un blocco sta cadendo, la sua posizione è prioritaria. Il modulo apprende automaticamente una *maschera* scalare per ogni cella, che amplifica le zone informative e sopprime quelle irrilevanti:

$$\mathbf{M} = \sigma(\mathrm{Conv}_{1\times1}(\mathbf{x})), \qquad \mathbf{x}_{\mathrm{att}} = \mathbf{x} \odot \mathbf{M} \tag{9}$$

   La convoluzione 1×1 proietta i 64 canali in un unico canale scalare per ogni posizione (i, j), che viene poi normalizzato in $[0, 1]$ dalla sigmoide. Questo meccanismo, ispirato alla Squeeze-and-Excitation [**?**] e al CBAM [**?**], è parametrico e differenziabile: la maschera non è fissa ma viene appresa durante il training, adattandosi dinamicamente alle caratteristiche del dominio.

4. Appiattimento in un vettore di $64 \times 11 \times 9 = 6336$ elementi.

Ramo scalare (stato interno). Il vettore $\mathbf{v} \in R^{32}$ viene processato da due layer `Linear` con attivazioni `ReLU`:

$$R^{32} \xrightarrow{W_1, b_1} R^{128} \xrightarrow{\mathrm{ReLU}} R^{128} \xrightarrow{W_2, b_2} R^{256} \xrightarrow{\mathrm{ReLU}} R^{256} \tag{10}$$

La scelta di un ramo separato (anziché concatenare il vettore scalare direttamente all'input del branch convoluzionale) è motivata dalla natura eterogenea dei due tipi di input: le convoluzioni sono ottimizzate per sfruttare la struttura spaziale locale, mentre le informazioni scalari non hanno struttura spaziale. Trattarle separatamente con layer lineari permette di apprendere interazioni non lineari tra le variabili di stato interno prima della fusione, evitando che il branch convoluzionale "sprechi" capacità cercando di decodificare feature senza struttura spaziale. L'espansione a 256 dimensioni garantisce che l'embedding scalare abbia una rappresentatività comparabile con quello visivo nella fase di fusione.

Fusione e testa Dueling. I due embedding vengono concatenati in un vettore combinato di dimensione $6336 + 256 = 6592$. Su questo vettore operano due teste separate, secondo l'architettura *Dueling DQN* [13].

L'idea fondamentale del Dueling DQN è la seguente: per molti stati, il valore di un'azione rispetto alle altre è simile (es. se stai cadendo nel vuoto, nessuna azione ti

salva), mentre il valore assoluto dello stato è basso. Unire queste due informazioni in un unico output Q non permette alla rete di distinguerle. Separando esplicitamente la stima del valore dello stato $V(s)$ ("quanto è buono trovarsi in questo stato?") da quella del vantaggio $A(s, a)$ ("quanto è migliore questa azione rispetto alle altre in questo stato?"), la rete può aggiornare $V(s)$ anche quando l'azione eseguita non è informativa sul vantaggio relativo, accelerando significativamente la convergenza:

- **Testa valore** $V(s)$: $R^{6592} \xrightarrow{\text{NoisyLinear}} R^{512} \xrightarrow{\text{ReLU}} R^1$
- **Testa advantage** $A(s, a)$: $R^{6592} \xrightarrow{\text{NoisyLinear}} R^{512} \xrightarrow{\text{ReLU}} R^6$

Il valore Q finale è calcolato secondo la formulazione standard:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right) \tag{11}$$

La centralizzazione del vantaggio (sottrarre la media) è necessaria per garantire l'identificabilità della decomposizione: senza di essa, V e A potrebbero compensarsi arbitrariamente (es. $V = 10, A = -10$ oppure $V = 0, A = 0$ producono lo stesso Q), rendendo instabile il training.

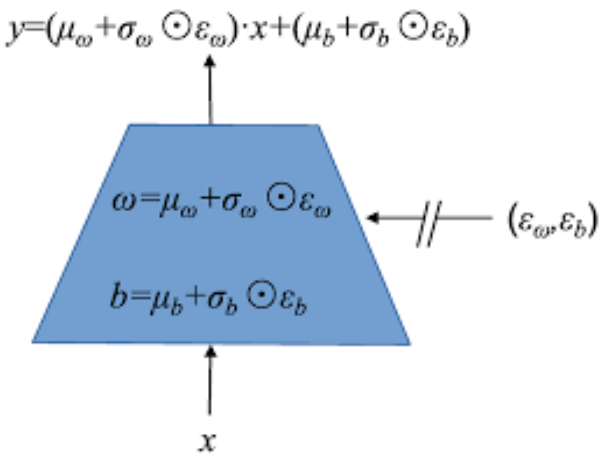


Figura 5: Funzionamento di un Noisy Layer

Noisy Nets per l'esplorazione. I layer `NoisyLinear` implementano la tecnica delle *Noisy Networks* [8]. Nel DQN classico, l'esplorazione avviene tramite la strategia ε-greedy: con probabilità ε si sceglie un'azione casuale. Questa tecnica ha però un limite fondamentale: il rumore è *esterno* alla policy e indipendente dallo stato. L'agente agisce casualmente con la stessa probabilità che si trovi in una regione del tutto inesplorata o in uno stato già ben conosciuto, sprecando molti step di esplorazione su percorsi già noti.

Le Noisy Networks risolvono questo problema rendendo il rumore *intrinseco* ai pesi della rete: ogni singolo peso viene parametrizzato come:

$$w_{ij} = \mu_{ij} + \sigma_{ij} \cdot \varepsilon_{ij} \tag{12}$$

dove μ_{ij} è il peso deterministico apprendibile, σ_{ij} è un parametro apprendibile che controlla l'"ampiezza" del rumore (anch'esso ottimizzato tramite backpropagation) e ε_{ij} è rumore

stocastico campionato da una distribuzione normale. Il risultato è che la rete stessa impara *quando* essere rumorosa: nelle regioni dello spazio degli stati già ben coperte, σ tende a 0 e la policy diventa deterministica; nelle regioni inesplorate, σ rimane alto mantenendo un'esplorazione attiva. Per ridurre il costo computazionale da $\mathcal{O}(nm)$ a $\mathcal{O}(n+m)$, viene adottato il *rumore fattorizzato*:

$$\varepsilon_{ij} = f(\varepsilon_i^{(i)}) \cdot f(\varepsilon_j^{(j)}), \quad f(x) = \mathrm{sgn}(x)\sqrt{|x|} \tag{13}$$

Anziché campionare un rumore indipendente per ogni coppia (i, j), si campionano solo n valori per gli input e m per gli output, combinandoli tramite prodotto tensoriale. La funzione $f(x) = \mathrm{sgn}(x)\sqrt{|x|}$ trasforma la distribuzione del rumore in modo da approssimare meglio una gaussiana standard nel prodotto.

L'inizializzazione dei pesi segue la strategia di He [9]: $\mathrm{Var}(w) = 2/n_{\mathrm{in}}$, ottimale per reti con attivazioni ReLU.

3.3 Algoritmo di Apprendimento: Rainbow DQN

Double DQN. Il DQN classico soffre di un problema di *sovrastima sistematica* dei Q-values [**?**]: quando si seleziona l'azione di massimo Q per calcolare il target, si usa lo stesso parametro θ sia per selezionare l'azione che per valutarla. Il rumore nei pesi della rete tende a far sì che l'azione selezionata sia quella con la stima più ottimistica (non necessariamente la migliore), introducendo un bias positivo che si accumula nel tempo e destabilizza il training.

Il Double DQN [10] risolve il problema disaccoppiando selezione e valutazione: la rete online Q_θ seleziona l'azione migliore, ma il suo valore viene stimato dalla rete target $Q_{\bar{\theta}}$:

$$y_t = r_t + \gamma \cdot Q_{\bar{\theta}}\Big(s_{t+1}, \arg\max_{a'} Q_\theta(s_{t+1}, a')\Big) \tag{14}$$

Poiché le due reti hanno pesi diversi (la target è una copia vecchia della online), l'errore di stima che porta alla sovrastima viene eliminato. La rete target $Q_{\bar{\theta}}$ viene copiata dalla rete online ogni `TARGET_UPDATE` $= 1000$ step: un intervallo lungo garantisce stabilità del target (se aggiornato troppo frequentemente diventa inutile), ma non così lungo da rallentare l'apprendimento.

Prioritized Experience Replay (PER). Nel DQN classico, le esperienze vengono campionate uniformemente dal replay buffer, con probabilità uguale per ogni transizione indipendentemente da quanto sia stata informativa. Questo è inefficiente: molte transizioni banali (es. perforare un blocco normale quando non ci sono minacce) vengono ripetute inutilmente, mentre le transizioni rare ma preziose (es. evitare per la prima volta un blocco in caduta) vengono viste raramente.

Il PER [11] assegna a ogni transizione una probabilità di campionamento proporzionale all'entità del suo errore temporale (*TD error* $|\delta_i|$): le transizioni su cui la rete è più "sorpresa" vengono campionate più spesso, accelerando l'apprendimento:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad p_i = |\delta_i| + \varepsilon \tag{15}$$

Il parametro $\alpha = 0.4$ controlla il grado di prioritizzazione: $\alpha = 0$ corrisponde al campionamento uniforme, $\alpha = 1$ dà assoluta priorità alle transizioni con errore maggiore. Il termine $\varepsilon = 10^{-5}$ garantisce che nessuna transizione abbia probabilità nulla di essere selezionata.

Poiché il campionamento non uniforme introduce un bias nella stima del gradiente (le transizioni frequentemente campionate pesano di più nell'aggiornamento), è necessario correggerlo tramite pesi di *importance sampling*:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^{\beta}, \quad \beta : 0.4 \to 1.0 \text{ in } 200\,000 \text{ frame} \tag{16}$$

β viene annealed da 0.4 a 1.0 durante il training: nelle fasi iniziali una correzione parziale è accettabile (i Q-values sono comunque imprecisi), mentre nella fase finale è importante garantire la correttezza del gradiente. La struttura dati sottostante è un **Segment Tree** (SumTree) che permette aggiornamento e campionamento in $\mathcal{O}(\log N)$, indispensabile per un buffer di grandi dimensioni. La loss pesata è:

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} w_i \cdot \mathcal{L}_{\text{Huber}}(Q_\theta(s_i, a_i),\ y_i) \tag{17}$$

La *Huber loss* è preferita alla MSE perché è meno sensibile agli outlier: per errori piccoli si comporta come la MSE (quadratica, gradienti proporzionali all'errore), mentre per errori grandi diventa lineare, evitando che singole transizioni con TD error molto alto producano aggiornamenti destabilizzanti.

N-step Returns. Nel DQN standard, il target di apprendimento usa solo la reward al passo successivo (r_t), propagando indirettamente il segnale attraverso i valori Q stimati. Questo rallenta enormemente il propagarsi del segnale di reward: per imparare che una sequenza di azioni porta alla vittoria del livello (evento raro), la rete deve imparare prima che l'ultima azione prima della vittoria è buona, poi che la penultima lo è, e così via, richiedendo potenzialmente centinaia di episodi.

Gli N-step Returns accelerano questa propagazione accumulando la reward su $N = 8$ step prima di calcolare il target:

$$G_t^{(N)} = \sum_{k=0}^{N-1} \gamma^k r_{t+k} \tag{18}$$

Con $N = 8$, ogni aggiornamento porta informazione su un orizzonte temporale 8 volte più lungo rispetto al DQN standard, permettendo al segnale di reward di propagarsi molto più rapidamente. Il valore $N = 8$ è un compromesso: valori troppo grandi aumentano la varianza del target (perché include più step futuri, ognuno dei quali è rumoroso) e possono essere problematici con un buffer di esperienze (le policy vecchie potrebbero aver generato sequenze non più rappresentative).

Ottimizzazione. L'ottimizzatore scelto è `AdamW` [12], una variante di Adam in cui il weight decay è applicato direttamente ai pesi (anziché essere incluso nel gradiente come in Adam), garantendo una regolarizzazione più corretta. I parametri $\eta = 2 \times 10^{-5}$, $\lambda = 10^{-5}$ e `amsgrad=True` bilanciano velocità di convergenza e stabilità: il learning rate basso previene oscillazioni sui Q-values; amsgrad mantiene il massimo del gradiente quadratico nel tempo, evitando che l'ottimizzatore dimentichi aggiornamenti critici. Uno scheduler coseno con $T_{\max} = 500\,000$ step riduce gradualmente il learning rate verso $\eta_{\min} = 10^{-6}$, permettendo un fine-tuning progressivo della policy nelle fasi avanzate del training. Infine, il gradient clipping a norma 1.0 previene l'esplosione del gradiente, fenomeno particolarmente problematico con il PER (dove alcune transizioni hanno pesi di IS molto alti).

3.4 Ciclo Decisionale: ACTION–WAITING–LEARNING

L'interazione tra agente e ambiente segue una macchina a stati a tre fasi. Questa struttura è stata necessaria perché il motore di gioco di *Mr. Driller* è *asincrono* rispetto all'agente: quando il giocatore perfora un blocco, la sua animazione richiede diversi frame prima di completarsi, durante i quali l'agente non dovrebbe prendere nuove decisioni (farlo porterebbe all'accumulo di comandi ignorati dal motore fisico). Il ciclo AWL sincronizza l'agente con il motore di gioco:

1. **ACTION**: l'agente è nello stato idle. La rete viene interrogata (o si campiona un'azione casuale con probabilità ε) e l'azione viene eseguita. Lo stato pre-azione $(s_t, \mathbf{v}_t, a_t)$ viene salvato per costruire la transizione al termine del ciclo.

2. **WAITING**: il motore fisico elabora le conseguenze dell'azione. L'agente attende che si verifichino simultaneamente due condizioni: il personaggio torna in stato idle *e* almeno un tick di gravità è trascorso. La seconda condizione è essenziale: senza di essa, la transizione verrebbe chiusa prima che la fisica del gioco (caduta di blocchi, effetto delle combo) abbia prodotto le sue conseguenze complete, rendendo la reward associata alla transizione incompleta e potenzialmente fuorviante.

3. **LEARNING**: vengono raccolti lo stato post-azione, la reward calcolata e il flag terminale. La transizione viene inserita nell' `NStepBuffer` (che accumula le reward N-step) e, quando il buffer è pronto, nel replay buffer principale. Ogni `TRAIN_EVERY` = 3 step viene eseguita un'iterazione di `optimize_model`. Questa frequenza è un compromesso: aggiornare ad ogni step renderebbe il training troppo correlato con le esperienze recenti; aggiornare troppo raramente rallenta la convergenza.

3.5 Funzione di Reward

La progettazione della funzione di reward è stata la componente più delicata dell'intero progetto, nonché quella che ha richiesto le iterazioni più numerose. Un agente che si limita a massimizzare il punteggio di gioco (l'unico segnale di reward "naturale" disponibile) tenderebbe a comportamenti miopi e all'ignoranza sistematica dell'ossigeno: nelle fasi iniziali del training, l'agente non collezionava mai capsule di ossigeno perché la reward per non morire era troppo ritardata e slegata dall'azione di raccolta. Il problema della *reward sparsa* in ambienti con orizzonte temporale lungo è un nodo centrale nel Deep RL [14]: è stato quindi necessario definire una reward densa e multidimensionale che guidasse il comportamento nella direzione desiderata.

Un principio fondamentale nella progettazione è stato quello di rendere ogni componente di reward il più possibile *causalmente legata* all'azione che la genera, minimizzando il lag temporale tra azione e feedback. Tutte le reward sono clampate nell'intervallo $[-12.0, +12.0]$ per evitare che singoli eventi eccezionali (es. una grande combo) producano aggiornamenti che sbilanciamo l'apprendimento delle policy per eventi più comuni.

3.5.1 Componenti Terminali

Vittoria.

$$r_{\text{win}} = 80.0 + v \cdot 15.0 + \frac{o}{100} \cdot 20.0 \tag{19}$$

La reward di vittoria non è fissa ma dipende dallo stato al momento del completamento: le vite residue v premiano una strategia conservativa che evita i rischi inutili; il livello di

ossigeno o al completamento premia l'efficienza nella gestione delle risorse. Senza questi bonus, un agente potrebbe imparare a vincere i livelli sistematicamente con 1% di ossigeno, un comportamento fragile che non generalizza ai livelli successivi del curriculum.

Morte.

$$r_{\text{death}} = -15.0 - \frac{R - y_t - 1}{\max(R-1, 1)} \cdot 25.0 \quad [-10.0 \text{ aggiuntivi se game over}] \tag{20}$$

La penalità di morte è proporzionale alla profondità mancante: morire al 10% della discesa è peggio che morire al 90%. Questo incentiva l'agente a "arrivare il più in fondo possibile", anche negli episodi in cui la morte è inevitabile, evitando il comportamento degenere di "arrendersi" e stazionare nella zona sicura in cima al livello.

3.5.2 Componenti Non-Terminali

Gestione della minaccia imminente. Questa componente di reward è nata dall'osservazione che l'agente, nelle prime fasi del training, continuava a perforare verso il basso anche quando un blocco stava cadendo direttamente sopra la sua testa, venendo sistematicamente schiacciato. Il problema era che il segnale di reward positivo per il progresso verticale era più forte del segnale negativo (differito) della morte. La penalizzazione diretta dei comportamenti errati in presenza di minaccia imminente ha risolto questo pattern:

- **Minaccia attiva** (blocco in caduta sopra): IDLE viene penalizzata di -2.0 (stare fermi sotto un blocco in caduta è letale); la perforazione laterale riceve -3.0 (spostarsi lateralmente *mentre* si perfora è più lenta di un semplice movimento ed è quindi ancora più pericolosa); i movimenti laterali puri non sono penalizzati perché rappresentano la risposta corretta.
- **Minaccia latente** (blocco in tremore): le penalità sono ridotte a -1.0 e -1.5 per non sovra-penalizzare una situazione ancora gestibile.

Schivata di un blocco in caduta. Il bonus di $+8.0$ per una schivata riuscita (blocco in caduta sopra + movimento laterale) ha avuto un impatto straordinario sull'apprendimento della sopravvivenza. Prima della sua introduzione, l'agente imparava a "non fare errori" (veniva penalizzato per i comportamenti sbagliati) ma non imparava a "fare la cosa giusta" in modo proattivo. Il rinforzo positivo diretto per il comportamento di evasione ha accelerato di molto la convergenza di questo comportamento critico.

Progresso verticale.

$$r_{\text{depth}} = \Delta y \cdot 2.5 + \frac{y_t}{R} \cdot 0.5 \quad (\text{se } \Delta y > 0) \tag{21}$$

La reward per la discesa è composita: il termine $\Delta y \cdot 2.5$ premia direttamente ogni passo verso il basso, mentre il termine $y_t/R \cdot 0.5$ è un bonus di "profondità assoluta" che cresce nel tempo, incentivando l'agente a continuare a scendere anche in zone già coperte. Un bonus addizionale di $+0.6$ premia specificamente l'azione `DRILL_DOWN`: senza di esso, l'agente tendeva a preferire percorsi laterali per evitare blocchi difficili, rallentando inutilmente la discesa.

Raccolta dell'ossigeno.

$$r_{\text{oxy}} = 3.0 + \left(1 - \frac{o_t}{100}\right) \cdot 17.0, \qquad r_{\text{oxy-deficit}} = -\frac{100 - o_t}{100} \cdot 0.15 \tag{22}$$

La reward per la raccolta dell'ossigeno è inversamente proporzionale al livello di ossigeno corrente: raccogliere una capsula quando si è all'1% vale ≈ 20.0 (urgenza massima), mentre raccoglierla al 100% vale solo 3.0 (potenzialmente uno spreco di posizione). Questo meccanismo insegna implicitamente all'agente la nozione di "urgenza relativa" della risorsa. Il termine di deficit $r_{\text{oxy-deficit}}$ è una piccola penalità continua per il basso livello di ossigeno, che mantiene la "preoccupazione" per l'ossigeno costante nel tempo piuttosto che manifestarsi solo al momento della raccolta.

Penalità per blocchi speciali. La penalità di -8.0 per i blocchi `unbreakable` è alta e immediata per garantire apprendimento rapido: ogni tentativo di perforare un muro indistruttibile è un'azione inutile che spreca un turno e non avanza il giocatore. I blocchi `delayed` vengono penalizzati di -3.0 solo in caso di tentativo *fallito* (il blocco non è ancora attivabile): questa penalità selettiva insegna all'agente a distinguere quando il blocco è perforabile da quando non lo è.

Reward shaping con PBRS [14]. Il Potential-Based Reward Shaping garantisce che il comportamento ottimale non cambi quando si aggiunge uno shaping reward:

$$F(s_t, s_{t+1}) = \gamma \cdot \frac{y_{t+1}}{R} \cdot 2 - \frac{y_t}{R} \cdot 2 \tag{23}$$

Questa componente non altera la policy ottimale (è una differenza di potenziali), ma velocizza significativamente la convergenza fornendo un segnale di reward densa che guida l'agente verso una maggiore profondità ad ogni step.

3.6 Sistema Anti-Loop: DrillTracker

Durante le prime fasi di training è emerso un comportamento degenere ricorrente: l'agente imparava a "oscillare" tra due posizioni adiacenti, accumulando piccole reward di movimento senza mai fare progresso reale. Questo fenomeno, noto nella letteratura come *reward hacking* [**?**], si verifica quando l'agente trova un modo per massimizzare la reward che non corrisponde all'obiettivo inteso dal progettista.

La classe `DrillTracker` rileva e penalizza questi pattern mantenendo una coda delle posizioni recenti:

$$r_{\text{osc}} = -\min(h \cdot 1.5,\ 6.0) \tag{24}$$

dove h è il numero di occorrenze della nuova posizione nella coda `recent_pos` di capacità 8. Visitare una posizione già vista di recente viene penalizzato in modo crescente con la frequenza di revisita, fino a un massimo di -6.0. Una penalità *column lock* di -2.0 viene applicata se l'agente staziona nella stessa colonna senza scendere per 12 step consecutivi (segnale di stagnazione verticale prolungata), con una segnalazione intermedia di -0.5 a 6 step che funge da "avvertimento precoce".

3.7 Gestione dell'Esplorazione: Epsilon Adattivo

La strategia di esplorazione combina tre meccanismi in modo sinergico:

1. **Noisy Nets**: il rumore parametrico descritto in Sezione 3.2 fornisce esplorazione intrinseca e adattiva anche con $\varepsilon = 0$, garantendo che l'esplorazione rimanga attiva nelle regioni dello spazio degli stati meno conosciute.

2. **ε-greedy a floor fisso**: si mantiene comunque un $\varepsilon_{\text{floor}} = 0.05$ come assicurazione contro situazioni in cui i Noisy Nets si siano "appiattiti" su una policy troppo deterministica, perdendo la capacità di esplorare nuove zone.

3. **Epsilon Boost adattivo**: ogni 2 000 step, se il win rate scende sotto 0.15 (l'agente non riesce a completare un livello su sette tentativi recenti), ε viene temporaneamente portato a 0.15. Questo meccanismo di auto-correzione contrasta i plateau di training: quando l'agente è bloccato in un ottimo locale, aumentare temporaneamente l'esplorazione casuale può permettergli di "uscire dal buco" e trovare nuove traiettorie di successo.

3.8 Fase di Warmup e Curriculum Learning

Prima del training vero e proprio, l'agente attraversa una fase di warmup di `WARMUP_STEPS` = 2 000 step con azioni casuali. Questo passo è necessario per garantire che il primo batch di training non sia campionato da una distribuzione degenere: se il replay buffer contenesse solo esperienze dello stesso tipo (es. tutte dall'inizio del primo livello), il primo aggiornamento dei pesi introdurrebbe un forte bias che può essere difficile da correggere. Il warmup con azioni casuali garantisce una copertura iniziale dello spazio degli stati sufficientemente diversificata.

Il *Curriculum Learning* [15], gestito dalla classe `CurriculumManager`, introduce la progressione graduale sui livelli. L'intuizione è analoga all'apprendimento umano: è più efficace padroneggiare prima i problemi semplici e poi affrontare quelli complessi, piuttosto che essere esposti subito alla complessità massima. I livelli avanzati di *Mr. Driller* hanno griglie più grandi, più blocchi speciali e ritmi di gioco più veloci: addestrarsi su di essi prima di aver consolidato le abilità di base porterebbe a un training lento e instabile. La soglia di win rate richiesta per avanzare al livello successivo cresce progressivamente, riflettendo l'aumento di difficoltà:

Tabella 5: Progressione del curriculum di addestramento.

Livello corrente	Livello da sbloccare	Win rate richiesto
1	3	$\geq 80\%$
3	5	$\geq 80\%$
5	7	$\geq 82\%$
7	10	$\geq 85\%$
10	— (training completo)	$\geq 90\%$

3.9 Persistenza e Monitoraggio del Training

L'intero stato del training è persistito su disco in checkpoint PyTorch (`.pth`) contenenti i pesi della rete, lo stato dell'ottimizzatore, lo stato dello scheduler, il contatore di step, ε

corrente e lo stato del `CurriculumManager`. La completezza del checkpoint è essenziale: salvare solo i pesi della rete non sarebbe sufficiente, perché lo stato interno di AdamW (i momenti di primo e secondo ordine) è necessario per riprendere il training da dove si era interrotto senza perdere la "memoria" dell'ottimizzatore. Il salvataggio avviene in un thread separato ogni 1 000 step per non bloccare il loop di training. Un meccanismo di *emergency save* intercetta il `KeyboardInterrupt` e salva l'ultimo checkpoint prima della chiusura, garantendo che nessuna sessione di training venga persa anche in caso di interruzione improvvisa.

Il monitoraggio delle metriche è delegato al modulo `TrainingMonitor`, che utilizza log CSV e grafici offline. La scelta di evitare TensorBoard è motivata da incompatibilità tecniche: TensorBoard utilizza gRPC e protobuf che entrano in conflitto con le librerie SDL/X11 usate da Pygame per il rendering del gioco. Le metriche comportamentali chiave tracciate ad ogni episodio includono il tasso di inattività (`IDLE_pct`), la frequenza di collisione con blocchi indistruttibili (`WallBash_Rate`) e la propensione alla perforazione verticale (`DrillDown_ratio`): questi indicatori forniscono una diagnosi comportamentale dell'agente indipendente dalla reward, permettendo di identificare problemi come l'eccessiva passività o la tendenza a "spaccare muri" prima che si riflettano nella curva di apprendimento.

4 Integrazione dei Large Language Models

L'integrazione di Large Language Models (LLM) all'interno di un sistema di gioco autonomo nasce da una considerazione fondamentale sul Deep RL: un agente tradizionale basato su reti neurali impara esclusivamente attraverso l'esperienza diretta, accumulando conoscenza in forma implicita nei propri pesi. Questo processo, pur efficace, richiede milioni di interazioni e non trasferisce facilmente conoscenze tra domini diversi.

Un LLM, al contrario, porta con sé una conoscenza pregressa acquisita durante il pre-addestramento su enormi corpora testuali. Questa conoscenza include principi generali di ragionamento strategico ("se una minaccia si avvicina, allora allontanarsi è prioritario rispetto all'obiettivo primario"), capacità di comprensione del linguaggio naturale e abilità di pianificazione a breve termine. La sfida è sfruttare questa conoscenza in un contesto real-time come un videogioco, dove la latenza di risposta è critica e lo stato deve essere tradotto in un formato testuale comprensibile al modello.

Nel sistema sviluppato, gli LLM sono stati integrati secondo due paradigmi distinti e complementari. Nel primo, *LLM as Action Generator*, il modello sostituisce interamente la policy del giocatore artificiale: viene usato come agente zero-shot, senza alcuna fase di addestramento sul dominio specifico. Nel secondo, *LLM as Reward Model*, il modello non gioca direttamente ma valuta la qualità delle transizioni prodotte da un agente RL tradizionale, fornendo il segnale di rinforzo scalare che guida l'apprendimento. Questi due approcci esplorano le capacità degli LLM in ruoli qualitativamente diversi: agente vs. giudice.

Entrambi i componenti condividono la medesima infrastruttura basata su **OpenRouter**, un servizio di routing unificato che permette di interrogare modelli diversi tramite un'unica API compatibile con lo standard OpenAI. I modelli selezionati bilanciano capacità di ragionamento e latenza:

```
AVAILABLE_MODELS = {
    "llama":    "meta-llama/llama-3.1-8b-instruct",
```

```
    "deepseek": "deepseek/deepseek-chat"
}
DEFAULT_MODEL_KEY = "llama"
```

Llama 3.1-8B è preferito come modello predefinito per la sua bassa latenza (8 miliardi di parametri vs. i decine di miliardi di alternative più potenti) e la sua buona capacità di seguire istruzioni strutturate, essenziale per il parsing affidabile delle risposte. La comunicazione avviene tramite il client HTTP `httpx`, configurato con supporto HTTP/2 e connessioni *keepalive* persistenti (5 connessioni max, timeout 30 s).

4.1 LLM come Action Generator

Nel primo approccio, l'LLM assume il ruolo di policy dell'agente: ad ogni step riceve una rappresentazione testuale dello stato e restituisce l'indice dell'azione tra le sei disponibili. Il problema centrale di questo approccio è la *grounding problem*: come trasformare lo stato di un gioco arcade (una griglia bidimensionale di blocchi con proprietà fisiche) in un testo che un LLM possa interpretare correttamente, mantenendosi entro i limiti di token che non rallentino eccessivamente la risposta?

Traduzione dello stato visivo in rappresentazione testuale

La funzione `build_compact_prompt()` costruisce un prompt strutturato in tre livelli di astrazione crescente, dal più concreto al più interpretabile.

Livello 1 — Griglia simbolica localizzata. Viene estratta la stessa finestra 11×9 già usata per il tensore visivo (Sezione 3.1.1), ma ogni cella viene mappata in un simbolo ASCII mnemonico anziché in un valore numerico. Questa scelta non è banale: i LLM sono addestrati su testo, e una griglia di simboli ASCII è più facilmente interpretabile di una matrice di float. I simboli sono stati progettati per essere intuitivi anche senza consultare la legenda:

Simbolo	Significato
`@`	Posizione corrente del giocatore
`.`	Cella vuota percorribile
`1`	Blocco classico
`1*`	Blocco classico parte di un gruppo combo
`S`	Blocco solo
`O`	Capsula di ossigeno
`X`	Blocco costoso (ogni colpo −20% ossigeno)
`D`	Blocco delayed non attivo
`!`	Blocco delayed con timer attivo
`v`	Prefisso: blocco in caduta imminente
`#`	Pareti perimetrali o fuori mappa

Livello 2 — Metadati contestuali. La prima riga del prompt contiene informazioni numeriche compatte in formato chiave=valore: posizione assoluta (x, y), profondità relativa, livello di ossigeno con segnalatori espliciti di criticità (`!!CRITICAL` per `oxy < 20`, `!LOW` per `oxy < 40`), numero di vite, punteggio e ultima azione eseguita. I segnalatori di criticità

sono fondamentali: i LLM rispondono bene a segnali testuali forti ed espliciti, e l'esperienza ha mostrato che senza di essi il modello tendeva a non gestire correttamente le situazioni di ossigeno critico.

Livello 3 — Minacce immediate e risorse critiche. La funzione `_immediate_threats()` riporta i blocchi in caduta nell'intera finestra nella forma `fall(dx,dy,tipo)`, dove (dx, dy) è la posizione relativa al giocatore. La funzione `_nearest_pill_info()` esegue una ricerca in una finestra 25×25 per localizzare la capsula più vicina secondo la distanza di Manhattan. Questi due elementi costituiscono le informazioni più urgenti per la sopravvivenza e vengono presentati in modo compatto e immediatamente riconoscibile.

Formulazione del system prompt

Il system prompt è stato progettato per essere il più compatto possibile (riduce il numero di token da processare) mantenendo al contempo la chiarezza delle istruzioni. Le priorità sono elencate esplicitamente in ordine decrescente di urgenza, rispecchiando la gerarchia di obiettivi del gioco:

```
"You are a Mr. Driller game agent. Select the best action.\n"
"SYMBOLS: @=you .=empty 1=classic O=oxygen X=costly S=solo D=delayed\n"
"ACTIONS: 0=IDLE 1=LEFT 2=RIGHT 3=DRILL_L 4=DRILL_R 5=DRILL_D\n"
"PRIORITY RULES:\n"
"  1. v in window -> move sideways to avoid crush\n"
"  2. oxy<20% -> reach nearest O capsule immediately\n"
"  3. oxy<40% -> plan path to nearest O capsule\n"
"  4. DRILL_D to go deeper (main objective)\n"
"  5. X blocks: NEVER drill into X | each hit costs 20% oxy\n"
"  6. Drill classic/solo blocks to clear path downward\n"
"  7. Avoid drilling delayed (D/!) blocks\n"
"  8. * combo: chain-destroys the whole color group\n"
"  9. IDLE only if moving would cause a falling block to hit you\n"
'Reply ONLY with compact JSON: {"action":<0-5>,"reason":"<max 8 words>"}'
```

Richiedere una risposta JSON compatta anziché del testo libero è fondamentale per due ragioni: il parsing è deterministico e veloce, e la struttura {`"action": N, "reason": "..."`} incoraggia il modello a esplicitare brevemente il ragionamento, migliorando la coerenza delle decisioni grazie all'effetto chain-of-thought implicito.

Ottimizzazioni implementate per la riduzione della latenza

La latenza è il principale ostacolo all'impiego di LLM in giochi real-time: una risposta API tipica richiede 200–500ms, mentre un frame di gioco dura ∼16ms. Sono state adottate quattro strategie complementari per rendere la latenza compatibile con il gameplay.

Cache LRU e Discretizzazione dello Stato. La funzione `_cache_key()` genera un hash MD5 della finestra 11×9, con ossigeno quantizzato al bucket di 10 e punteggio in centinaia. Questa discretizzazione riduce il numero di stati "distinti" di ordini di grandezza: stati che differiscono solo per variazioni minori (es. ossigeno al 61% vs 62%) vengono trattati come identici, il che è ragionevole dato che la policy ottimale non cambia per

variazioni così piccole. La policy LRU è realizzata con complessità $\mathcal{O}(1)$ sfruttando l'ordine di inserimento dei dizionari Python 3.7+. La cache è dimensionata per `CACHE_SIZE` = 512 entry: un valore più grande avrebbe ridotto i cache miss ma aumentato la memoria utilizzata e il rischio di servire risposte obsolete.

Streaming SSE con Early Exit JSON. Anziché attendere il completamento della risposta, il buffer di testo viene analizzato ad ogni chunk ricevuto via Server-Sent Events. Non appena la regex `\{"action"\s*:\s*(\d)[^}]*\}` individua il campo `action`, lo stream viene interrotto anticipatamente, risparmiando il tempo di trasmissione dei token successivi (la spiegazione, i token di chiusura). Nella pratica, questo riduce la latenza effettiva del 30–50% rispetto all'attesa del completamento.

HTTP/2 e Connection Pooling. L'uscita anticipata dallo stream non causa la chiusura del socket fisico grazie al multiplexing HTTP/2: il client invia un frame `RST_STREAM` senza chiudere la connessione, che rimane disponibile nel pool keepalive per la richiesta successiva. Senza HTTP/2, ogni richiesta richiederebbe un nuovo TCP handshake (∼50ms aggiuntivi per connessioni intercontinentali).

Retry con back-off e gestione del rate limiting. In caso di risposta nulla, il sistema esegue fino a `MAX_RETRIES` = 2 tentativi con intervallo fisso di 0.3 s. Le risposte HTTP 429 (rate limit superato) vengono gestite leggendo l'header `Retry-After` e aggiungendo un margine di sicurezza di 1 secondo. L'azione di fallback predefinita è `DRILL_D` (indice 5): in caso di mancata risposta, è più sicuro tentare di avanzare verso il basso (l'obiettivo principale) che restare fermi o muoversi lateralmente in modo casuale.

4.2 LLM come Reward Model

Il secondo approccio adotta una prospettiva radicalmente diversa rispetto all'LLM as Action Generator. Anziché usare il modello come policy diretta, esso viene impiegato come componente di *valutazione semantica*: per ciascuna transizione (s_t, a_t, s_{t+1}) prodotta dall'agente RL, l'LLM viene interrogato per stimare un valore scalare di rinforzo r_t. Questo paradigma, ispirato al campo dell'*RLHF* (Reinforcement Learning from Human Feedback), delega all'LLM il compito di interpretare il contesto dell'azione, un compito che richiede comprensione semantica difficile da codificare con regole procedurali.

Costruzione del prompt di transizione

La funzione `build_compact_reward_prompt()` costruisce una rappresentazione sintetica su tre righe. A differenza del prompt per l'Action Generator (che deve trasmettere l'intera griglia visiva), questo prompt si concentra sulle *variazioni differenziali* rilevanti per la valutazione: Δy (progresso verticale), Δoxy (variazione ossigeno), Δscore (variazione punteggio) e gli eventi speciali (`LEVEL_COMPLETE`, `LIFE_LOST`, `HIT_X`, `NOT_MOVED`). Il formato è estremeamente compatto:

```
action=DRILL_D dy=+1 pos=(4,18)->(4,19)
oxy=62%->60%(d-2%) score=240->290(d+50) depth=19.0%
events:none
```

Questa sintesi è sufficiente per valutare la transizione: un LLM può dedurre che perforare verso il basso ($\Delta y = +1$) a un livello di ossigeno confortevole (62%) con un guadagno di punteggio (+50) è una transizione positiva, senza bisogno di vedere l'intera griglia.

Funzione di reward e criteri di valutazione

Il system prompt definisce esplicitamente la funzione di utilità che l'agente deve massimizzare. La motivazione principale per usare un LLM invece di un semplice script procedurale risiede nella regola semanticamente complessa di penalizzare l'`IDLE ingiustificata`: mentre una funzione hard-coded rileva solo che l'agente è rimasto fermo, l'LLM è in grado di interpretare l'intero contesto dello stato per dedurne la motivazione. Se l'agente è rimasto fermo perché c'era un blocco in caduta sopra di lui (`IDLE` come strategia di evasione), la penalità non deve essere applicata; se invece è rimasto fermo senza alcuna minaccia visibile, è un'inazione inefficiente che va scoraggiata. Una tale discriminazione richiede un'analisi contestuale che va oltre le capacità di un semplice sistema a regole.

Evento	**Contributo al reward**
Discesa di una cella ($\Delta y > 0$)	+5 per cella
Guadagno di ossigeno (spec. se oxy $< 40\%$)	+2 per % guadagnata
Incremento del punteggio	+10 per 100 punti
Perdita di una vita	-100
Completamento del livello	+300
Tentativo su blocco costoso (`HIT_X`)	-10
Azione `IDLE` ingiustificata	-0.3

Il valore scalare restituito viene moltiplicato per `REWARD_SCALE` $= 1.0$ e convertito in un `torch.Tensor` prima di essere inserito nel replay buffer. In caso di mancata risposta (timeout, errore API), viene attivato un *fallback gerarchico*: prima la funzione di reward deterministica standard di `ai_agent` (garantendo che l'agente non riceva un segnale nullo che azzererebbe il segnale di apprendimento), poi il valore neutro `FALLBACK_REWARD` $= 0.0$ come ultima risorsa.

4.3 Framework di Benchmarking Comparativo

Il modulo `benchmark.py` implementa un framework di valutazione sistematica che permette di confrontare le prestazioni operative dei diversi LLM su stati e transizioni *mock* generati proceduralmente, senza richiedere sessioni di gioco reali. Questo è particolarmente utile per stimare il trade-off latenza/qualità tra modelli prima di un'integrazione full-scale: testare l'Action Generator in gioco richiederebbe ore di sessione con il modello sotto test, mentre il benchmark restituisce metriche affidabili in pochi minuti.

Il punteggio composito $S \in [0, 100]$ combina quattro metriche operative con pesi che riflettono le priorità per un agente real-time:

$$S = S_{\text{lat}} + S_{\text{err}} + S_{\text{cons}} + S_{\text{cache}} \tag{25}$$

$$S_{\text{lat}} = 40 \cdot \max\left(0,\ 1 - \frac{\ell}{2000}\right), \quad S_{\text{err}} = 30 \cdot (1 - \min(\varepsilon, 1)) \tag{26}$$

$$S_{\text{cons}} = 20 \cdot c, \quad S_{\text{cache}} = 10 \cdot h \tag{27}$$

Il peso del 40% assegnato alla latenza (S_{lat}) riflette la sua importanza primaria per l'usabilità real-time: un modello molto intelligente ma con 3 secondi di latenza media è inutilizzabile in un gioco arcade. Il termine di consistenza ($S_{\text{cons}}, 20\%$) misura la ripetibilità delle risposte a prompt identici, cruciale per la stabilità del comportamento. Il term di cache hit rate ($S_{\text{cache}}, 10\%$) premia i modelli che producono risposte più facilmente cacheabili (ovvero più deterministiche), riducendo il numero di chiamate API effettive durante il gioco.

5 Sperimentazioni, Sfide e Risultati

5.1 Agente Rainbow DQN: dal Prototipo alla Versione Finale

Lo sviluppo dell'agente Rainbow DQN ha attraversato un percorso lungo e non lineare, fatto di iterazioni, regressioni e correzioni progressive. I tre problemi principali che hanno dominato l'intero ciclo di sviluppo sono stati: la progettazione di una funzione di reward adeguata, la corretta modellazione della fisica asincrona del gioco all'interno di un framework RL, e le severe limitazioni computazionali dell'hardware disponibile. Ciascuno di questi aspetti ha richiesto interventi architetturali non banali e ha lasciato una traccia tangibile nelle scelte finali dell'agente.

5.1.1 Il problema della funzione di reward

La funzione di reward è stata, sin dall'inizio, la componente più critica dell'intero sistema. Una funzione di reward mal calibrata porta inevitabilmente a comportamenti patologici che possono persistere per milioni di step senza che l'agente mostri alcun segnale di convergenza verso la politica desiderata.

Il primo prototipo adottava una reward densa ma estremamente punitiva: ogni blocco forato nelle direzioni sbagliate, ogni oscillazione, ogni contatto con un blocco indistruttibile veniva sanzionato con penalità dell'ordine di -10 o -20. La varianza del segnale di reward era altissima, con alternanze rapide tra valori molto positivi (chain reaction di blocchi classic) e fortemente negativi (una singola morte). L'effetto pratico su una rete addestrata con PER è particolarmente insidioso: la Prioritized Experience Replay campiona preferenzialmente le transizioni con errore TD elevato, che in questo caso corrispondevano quasi sempre agli step di penalità massima. La rete si concentrava quindi su questi casi estremi, ignorando le transizioni più comuni che costituivano il flusso ordinario di gioco.

Ne derivava il primo ottimo locale macroscopicamente osservabile: l'agente imparava rapidamente che la strategia *meno rischiosa* era il drilling verticale ininterrotto verso il basso. Ogni deviazione laterale esponeva al rischio di colpire blocchi indistruttibili, di innescare chain reaction laterali pericolose, o di perdere tempo prezioso mentre l'ossigeno calava. Il drilling verticale, pur non essendo la politica ottimale, garantiva un flusso costante di reward positivo da progresso verticale con bassa varianza e basso rischio. L'agente si cristallizzava su questa soluzione sub-ottimale e non riusciva a uscirne perché le traiettorie alternative, necessariamente più rischiose nella fase di esplorazione, venivano sistematicamente penalizzate.

Il secondo fenomeno patologico osservato è stato l'*oscillazione laterale*. In alcune sessioni di training, l'agente scopriva che muoversi ripetutamente avanti e indietro tra due celle adiacenti neutralizzava la penalità dell'azione IDLE, pur senza produrre alcun

progresso reale. La causa profonda era una reward che penalizzava l'IDLE in modo isolato, senza penalizzare l'assenza di progresso verticale in quanto tale.

La soluzione finale è stata una funzione di reward multi-componente bilanciata, costruita per approssimazione progressiva. I componenti principali sono stati: una componente di progresso verticale proporzionale alla distanza dal fondo del livello (r_{depth}), una componente di deficit ossigeno continuo ($r_{\text{oxy-deficit}}$), una componente di raccolta capsula molto pronunciata e calibrata in funzione dell'urgenza (r_{oxy}, fino a +19.3 per raccolta a ossigeno quasi esaurito), e le penalità del sistema `DrillTracker` per oscillazioni e column lock. La riduzione progressiva delle penalità assolute, sostituita da penalità proporzionali al comportamento deviante, ha abbassato drasticamente la varianza del segnale senza perdere il potere informativo.

5.1.2 La fisica asincrona e il processo markoviano

Il secondo grande problema ha riguardato la corretta modellazione della fisica del gioco. *Mr. Driller* è un ambiente *intrinsecamente asincrono*: la gravità opera su un tick-loop indipendente dall'input del giocatore. Quando un blocco viene distrutto, i blocchi sovrastanti non cadono istantaneamente, ma attraversano una sequenza di stati fisici intermedi (`startShaking` → `isFalling` → impatto) distribuiti su più frame di gioco. Questo comportamento introduce una violazione della proprietà di Markov: lo stato osservabile al frame t non è sufficiente per prevedere deterministicamente lo stato al frame $t + 1$.

La prima conseguenza pratica fu che l'agente sembrava non riuscire ad apprendere la relazione causale tra le proprie azioni e la caduta dei blocchi. Un drilling aggressivo su un cluster classic innescava una chain reaction con reward positivo allo step t; la caduta letale si verificava allo step $t + 6$ o $t + 7$, ben oltre la finestra N-step allora impostata a $N = 5$.

La soluzione definitiva si articolò su tre livelli. A livello di buffer N-step, l'orizzonte fu portato da $N = 5$ a $N = 8$, portando la penalità di morte al passo $t+6$ all'interno della finestra di ritorno scontato con fattore $\gamma^6 \approx 0.941$. A livello percettivo, lo `SmartFrameStacker` fu configurato per accumulare $N_{\text{frames}} = 4$ frame consecutivi, rendendo implicita l'informazione sul movimento dei blocchi. A livello del tensore visivo, il Canale 3 fu ristrutturato per propagare esplicitamente il valore di urgenza verso il basso lungo la colonna del blocco in caduta. Questi tre interventi combinati hanno permesso di approssimare ragionevolmente la proprietà di Markov in un ambiente che strutturalmente non la soddisfa.

5.1.3 Sessione di Training e Dinamiche di Apprendimento

L'agente è stato addestrato per **536 400** step su hardware CPU commodity, in assenza di GPU dedicata. Il curriculum di training ha guidato l'agente dal livello 1 verso i livelli più avanzati. Il checkpoint finale (`driller_ai_v31.pth`) è stato utilizzato per la valutazione comparativa.

L'architettura Rainbow DQN si è rivelata robusta fin dalle prime sessioni di training. Il Double DQN ha prevenuto la sovrastima sistematica dei Q-value che affligge il DQN classico. Le Noisy Nets hanno consentito un'esplorazione più efficace e adattiva, concentrandola nelle regioni di maggiore incertezza. La PER ha permesso di campionare con frequenza maggiore le transizioni più informative, accelerando la convergenza. L'N-step Return buffer ha risolto il problema della propagazione dei reward ritardati.

L'analisi dei log di training tramite `TrainingMonitor` ha evidenziato le seguenti dinamiche. La metrica `DrillDown_ratio` ha iniziato intorno al 70–80% nelle prime fasi di curriculum (comportamento verticale sub-ottimale), scendendo progressivamente verso il

35–40% nella versione finale grazie al sistema DrillTracker. La causa principale di morte è rimasta il *block fall* (schiacciamento da blocchi cadenti) piuttosto che l'esaurimento dell'ossigeno, indicando che l'agente ha imparato a gestire le capsule ma fatica ancora con la fisica dei blocchi cadenti in scenari complessi.

La ricerca degli iperparametri ha richiesto numerose sessioni. Il batch size è stato aumentato da 64 a 128: con il blocco di spatial attention presente nell'architettura, un batch più ampio consente di osservare una maggiore varietà di configurazioni spaziali per iterazione. Il learning rate è stato ridotto da 6.25×10^{-5} all'attuale $\eta = 2 \times 10^{-5}$: le sessioni con learning rate più alto mostravano fasi di stagnazione interrotte bruscamente da crolli verso reward negativi, pattern tipico di un learning rate troppo alto in presenza di reward ad alta varianza.

Note sul training

A causa delle limitazioni hardware (assenza di GPU), il curriculum non ha potuto essere completato integralmente. L'agente ha mostrato convergenza sui livelli iniziali (win rate $\geq$80% su livello 1) ma può beneficiare significativamente di sessioni di training estese su GPU, che permetterebbero di raggiungere le soglie di win rate richieste per i livelli avanzati (85–90% su livello 10).

5.2 LLM Agent e LLM Reward Model: Ottimizzazione e Validazione

5.2.1 Il problema della latenza: dal prototipo all'agente ottimizzato

L'integrazione degli LLM nel loop di gioco ha immediatamente evidenziato una tensione fondamentale: la latenza di inferenza di un modello linguistico remoto è strutturalmente incompatibile con i requisiti temporali di un agente di controllo in tempo reale.

Il prototipo iniziale dell'LLM Action Generator non includeva alcuna ottimizzazione della comunicazione: ogni chiamata apriva una nuova connessione TCP, completava il TLS handshake, inviava la richiesta in attesa della risposta completa e chiudeva la connessione. La latenza media misurata si attestava stabilmente tra i 15 e i 18 secondi per chiamata, rendendo l'agente completamente inutilizzabile.

Il problema era aggravato dalla dimensione del prompt. Nelle prime versioni il prompt includeva l'intera finestra visiva 11×9 con simboli estesi, le regole complete del gioco, i metadati contestuali e le informazioni sulle minacce, per un totale di circa 400–500 token di input. Ridurre il prompt avrebbe abbassato la latenza di inferenza ma avrebbe privato il modello delle informazioni necessarie per comprendere la fisica del gioco. In particolare, un modello che riceveva solo i 4 vicini ortogonali del personaggio era cieco ai blocchi in tremore nelle righe superiori.

La strategia di ottimizzazione ha puntato a ridurre la latenza *di comunicazione* senza toccare la finestra di contesto. Le due ottimizzazioni che hanno prodotto l'impatto maggiore sono state il meccanismo di *streaming SSE con early JSON exit* (riduzione del 20–40% per le chiamate non in cache) e la migrazione a connessioni HTTP/2 keepalive (eliminazione del costo di TCP handshake, tipicamente 30–80 ms, e di TLS handshake, 100–200 ms aggiuntivi).

Il terzo intervento più impattante è stato la cache LRU su stato discretizzato. Nelle fasi di gioco in corridoi stretti o con dinamica ripetitiva, il tasso di cache hit raggiunge

valori molto elevati (93% nella valutazione reale), riducendo la latenza media percepita significativamente al di sotto di quella della singola chiamata API.

Il risultato complessivo ha portato la latenza media per azione da 15–18 secondi a 0.5–1.5 secondi.

5.2.2 Ingegnerizzazione del prompt e compromessi contestuali

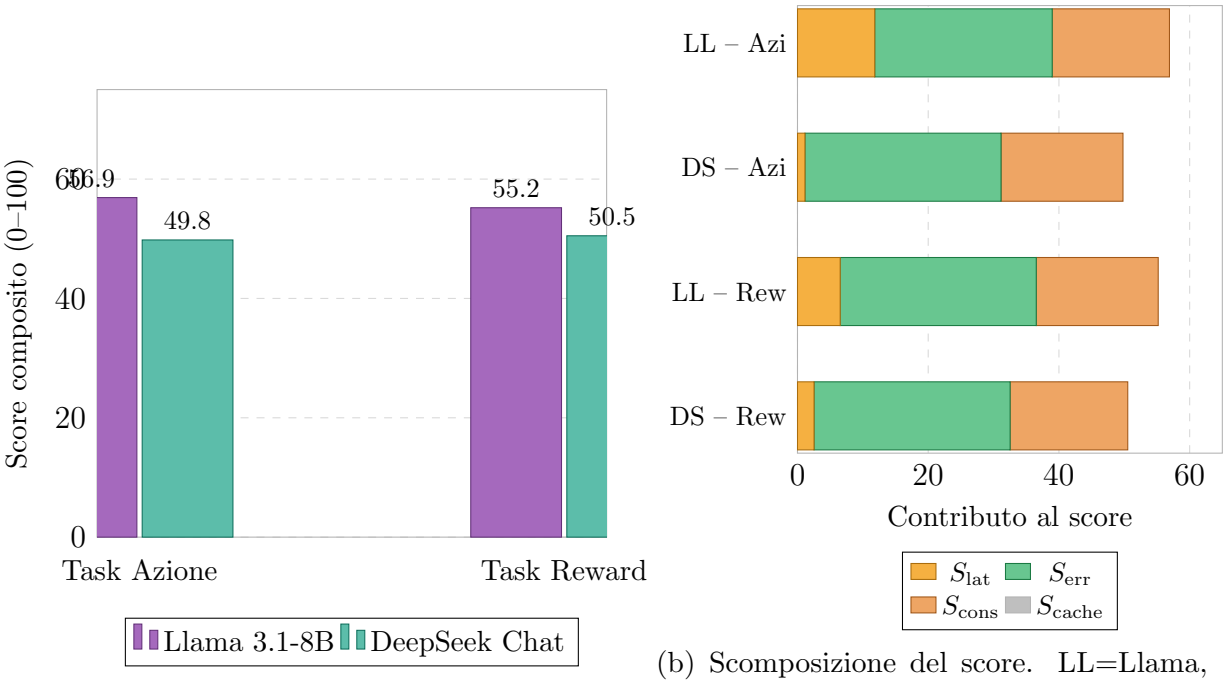
Il prompt engineering ha rappresentato un'attività iterativa quanto la progettazione della funzione di reward. Un problema specifico ha riguardato i blocchi *delayed*: nelle prime versioni del prompt venivano descritti come "esplosivi", generando nel modello l'aspettativa che dovessero essere evitati a tutti i costi. La formulazione corretta — "richiedono un solo colpo per avviare un timer di 3 secondi, poi spariscono senza causare danni" — ha prodotto una comprensione più accurata. Analoga revisione è stata necessaria per i blocchi unbreakable, inizialmente descritti come "indistruttibili" (tecnicamente impreciso: hanno HP=5 ma ogni colpo costa il 20% di ossigeno).

5.2.3 Risultati del Benchmark LLM

Il benchmark sistematico ha valutato i due modelli su 30 stati/transizioni mock per ciascun task. I risultati completi sono riportati in Tabella 6.

Tabella 6: Risultati del benchmark LLM sui task di selezione azione e calcolo reward (30 campioni per task, seed fisso per riproducibilità).

Task	Modello	Latenza (ms)	Cache Hit	Errori	Consistenza	Score
Azione	**Llama 3.1-8B**	**1 409**	0.0%	9.5%	89.7%	**56.9**
	DeepSeek Chat	1 943	0.0%	0.0%	93.3%	49.8
Reward	**Llama 3.1-8B**	**1 673**	0.0%	0.0%	93.3%	**55.2**
	DeepSeek Chat	1 874	0.0%	0.0%	90.0%	50.5



(a) Score composito per task e modello.

(b) Scomposizione del score. LL=Llama, DS=DeepSeek.

Figura 6: Risultati del benchmark LLM. Llama 3.1-8B vince in entrambi i task principalmente grazie alla latenza inferiore (S_{lat}: 11.82 vs 1.14 per il task Azione). DeepSeek è più affidabile (0% errori) ma la penalizzazione per latenza (quasi al limite di 2000 ms) non viene compensata.

Vincitore Benchmark

Llama 3.1-8B si classifica primo in entrambi i task con un punteggio medio di 56.0/100 (vs 50.2/100 di DeepSeek) ed è stato selezionato come modello predefinito per la valutazione finale. Il vantaggio è determinato principalmente dalla latenza di inferenza significativamente inferiore. Il componente S_{cache} è nullo per entrambi nel benchmark mock (stati non ripetitivi), diversamente dall'uso reale dove la cache LRU produce hit rate del 93%.

6 Valutazione Comparativa dei Tre Agenti

6.1 Metodologia di Valutazione: il Modulo `evaluate_agents.py`

La valutazione comparativa è stata condotta tramite lo script `evaluate_agents.py`, progettato per garantire condizioni di test identiche e riproducibili per tutti e tre i sistemi. La scelta di sviluppare un modulo di valutazione dedicato, separato dal loop di training e dal loop di gioco principale, risponde all'esigenza di disaccoppiare la fase di test da qualsiasi artefatto legato al training (policy in aggiornamento, replay buffer parzialmente riempito, curricula non completati).

L'ambiente di valutazione replica fedelmente il loop di gioco di `main.py` in modalità headless, con driver video e audio fittizi e timer virtuali. La macchina a stati interna è identica a quella del gioco reale: `ACTION` → `WAITING` → `ACTION`. I tre agenti confrontati sono: `dqn` (Rainbow DQN puro, checkpoint a 536 400 step), `llm_act` (Llama come policy diretta) e `llm_rew` (DQN addestrato con reward LLM, stesso checkpoint del DQN puro).

La valutazione si è svolta su **10 episodi** per agente, con seed fisso (`seed=42`) e bootstrap con 2 000 campioni per gli intervalli di confidenza al 95%.

Metriche raccolte

Le metriche operative coprono cinque dimensioni complementari:

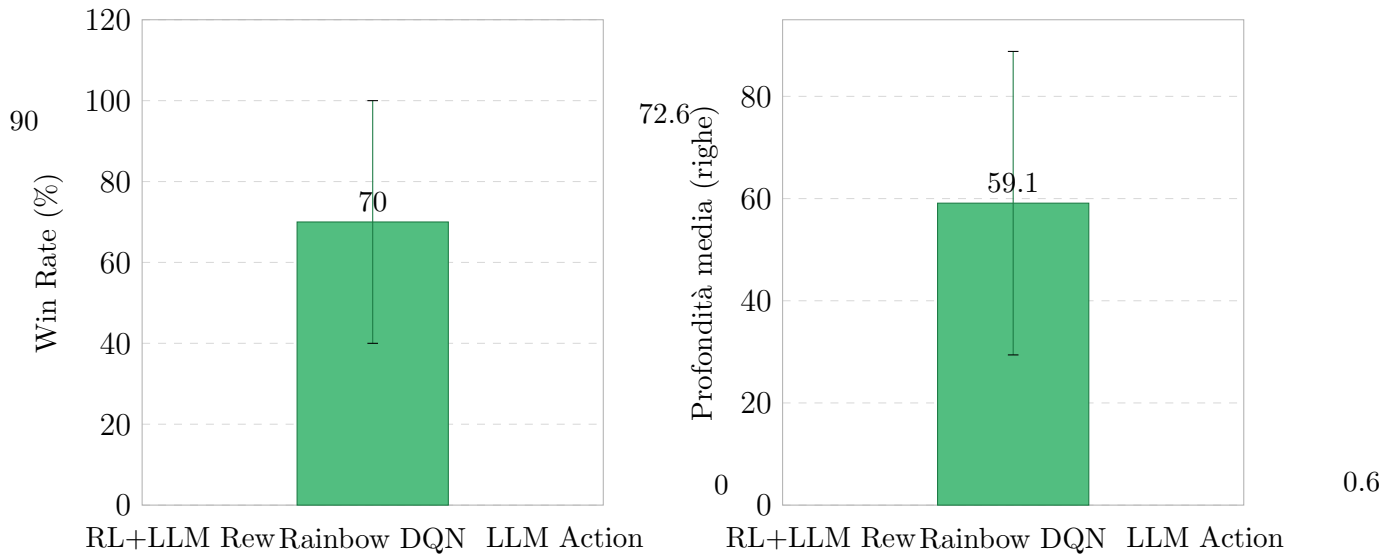
- **Risultato finale**: win rate, distribuzione delle cause di terminazione, vite consumate per episodio.
- **Progresso**: profondità massima raggiunta e profondità media su tutti gli step.
- **Efficienza**: step per episodio e azioni per unità di progresso verticale.
- **Gestione delle risorse**: ossigeno medio al termine, capsule raccolte, frequenza sotto soglia critica.
- **Qualità dell'inferenza** (solo LLM): chiamate API, cache hit, errori, latenza, consistenza.

6.2 Risultati Principali

La Tabella 7 riassume le metriche principali per ciascun agente.

Tabella 7: Sintesi dei risultati della valutazione comparativa (10 episodi per agente, CI 95% bootstrap con 2000 campioni). La metrica "Depth avg" indica la profondità media raggiunta in righe dalla cima del livello.

Agente	Win Rate	CI 95%	Depth avg±std	Steps avg±std	Score medio
RL+LLM Reward	**90.0%**	[70.0, 100.0]	72.6 ± 7.2	190.0 ± 115.3	**1 353**
Rainbow DQN	70.0%	[40.0, 100.0]	59.1 ± 29.7	198.2 ± 215.3	1 141
LLM Action	0.0%	[0.0, 0.0]	0.6 ± 0.9	112.7 ± 41.5	24



(a) Win Rate con CI 95% (bootstrap). (b) Profondità media con CI 95% (bootstrap).

Figura 7: Confronto di Win Rate e profondità media tra i tre agenti. Le barre di errore mostrano la deviazione standard. RL+LLM Reward domina su entrambe le metriche, con varianza notevolmente inferiore rispetto al Rainbow DQN puro.

Risultato principale

RL+LLM Reward ottiene il win rate più elevato (90%) e la maggiore profondità media (72.6 ± 7.2 righe), con la minore varianza tra gli agenti non banali (CV = 9.9% vs 50.3% del DQN puro). Il segnale semantico LLM produce policy più **robuste** e **consistenti** rispetto alla sola funzione di reward hand-crafted.

6.3 Cause di Fine Episodio

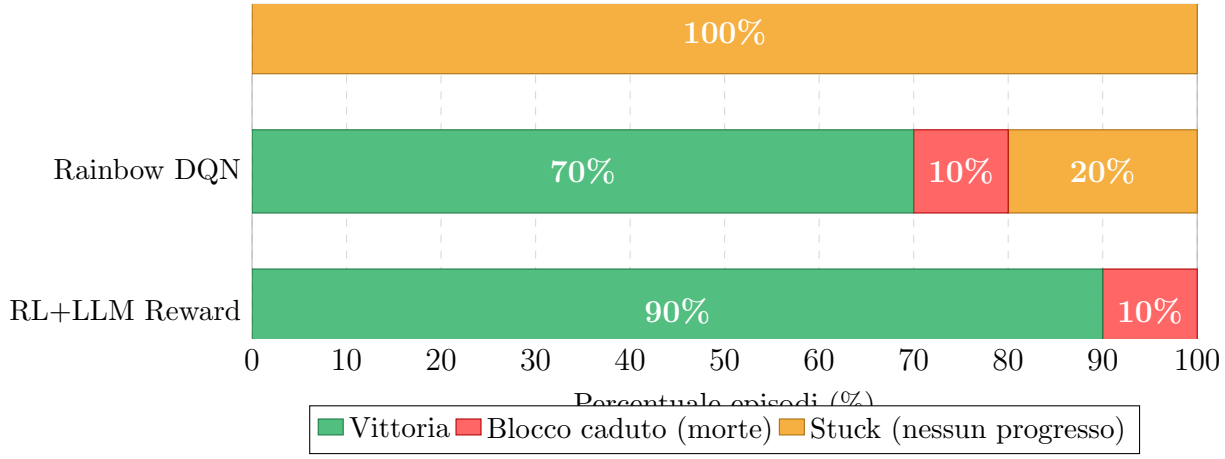


Figura 8: Distribuzione percentuale delle cause di fine episodio per i tre agenti. Il dato più rilevante è che l'LLM Action termina il **100%** degli episodi in stato *stuck*: l'agente non viene né schiacciato da blocchi né ucciso dall'ossigeno, ma rimane bloccato in pattern di drilling laterale senza mai scendere. Il Rainbow DQN ha il 20% di episodi stuck da loop di oscillazione non intercettati dal timeout. RL+LLM Reward non presenta episodi stuck (0%).

6.4 Distribuzione delle Azioni

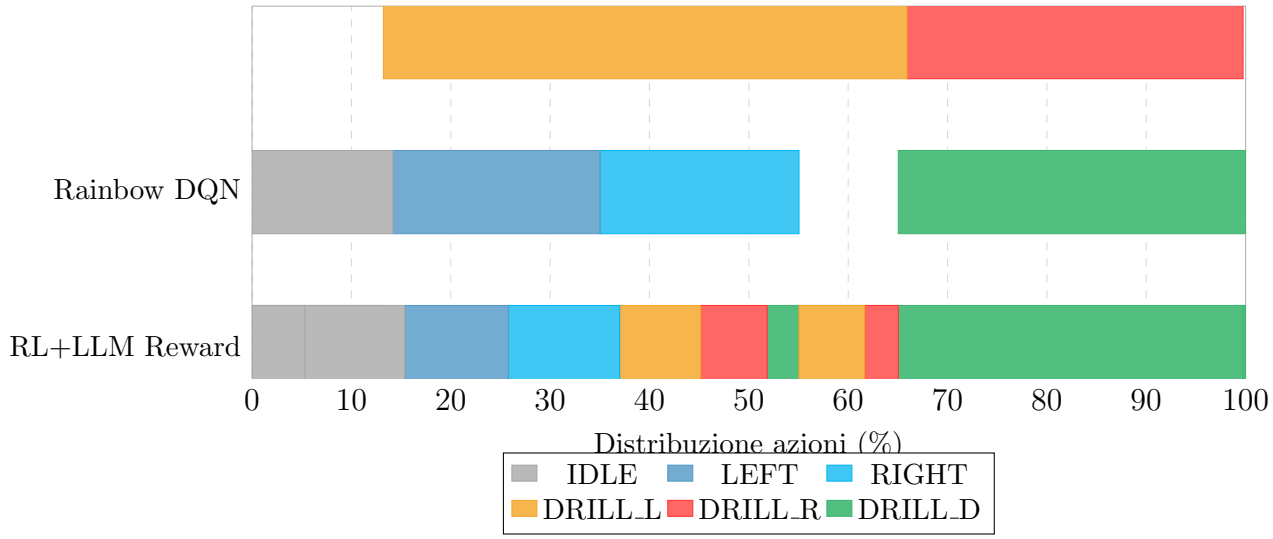


Figura 9: Distribuzione delle azioni intenzionate per i tre agenti. L'LLM Action usa DRILL_L (52.7%) + DRILL_R (33.8%) = **86.5%** di drilling laterale, con appena **0.27%** di DRILL_D: il modello interpreta correttamente le priorità del prompt ma le traduce in drilling laterale invece che verticale. RL+LLM Reward mostra il maggior utilizzo di DRILL_D (48.2%), coerente con l'obiettivo di discesa.

L'entropia di Shannon della distribuzione delle azioni rivela la diversità comportamentale:

- **Rainbow DQN**: 2.29 bit (distribuzione bilanciata tra più azioni)
- **RL+LLM Reward**: 2.17 bit (leggermente più focalizzato su DRILL_D)
- **LLM Action**: 1.63 bit (distribuzione altamente concentrata su DRILL_L/R)

6.5 Statistiche di Inferenza LLM durante la Valutazione

Tabella 8: Statistiche operative dell'LLM Action agent durante la valutazione (10 episodi). Il confronto con il benchmark mock evidenzia l'impatto della cache LRU in sessioni reali.

Metrica	**Valore**	**Nota**
Chiamate API effettuate	78	Con cache svuotata ad ogni episodio
Cache hit	1 054	Tasso: **93.1%**
Errori API	5	Tasso: 6.4% (su 78 chiamate)
Latenza media per azione	898.2 ms	Include streaming con early exit
Consistenza	100.0%	Stessi stati → stessa azione

Il tasso di cache hit del 93.1% durante la valutazione reale è ben superiore allo 0% del benchmark mock. Questo conferma che in sessioni di gioco reali gli stati si ripetono molto frequentemente (drilling in corridoi stretti, situazioni ricorrenti), rendendo la cache LRU

particolarmente efficace. La latenza effettiva media di 898 ms è inferiore ai 1 409 ms del benchmark, proprio perché la maggioranza delle chiamate viene servita dalla cache in tempo costante.

6.6 Analisi Comparativa Complessiva

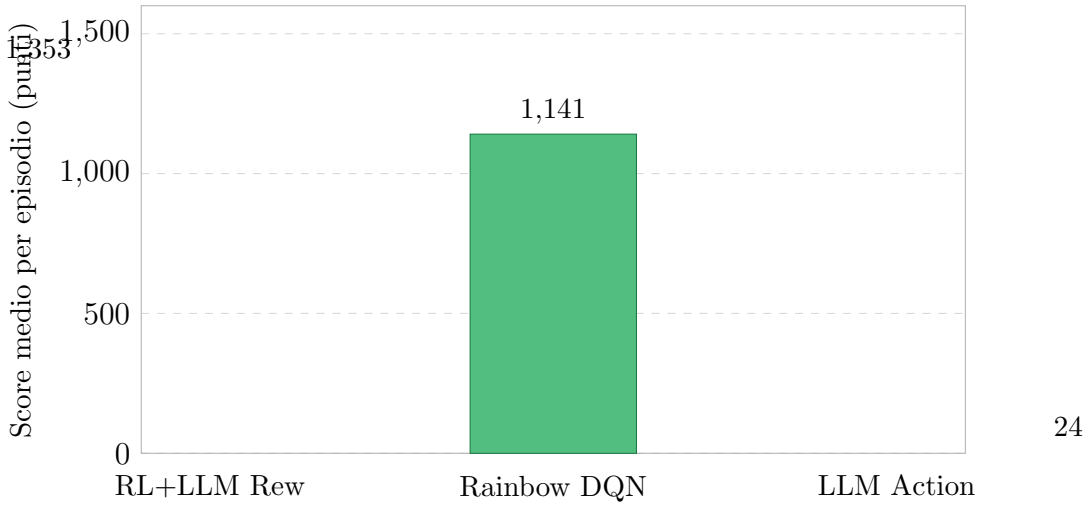


Figura 10: Score medio di gioco per episodio (punteggio ottenuto rompendo blocchi). RL+LLM Reward eccelle sia in win rate che in punteggio; l'LLM Action ottiene appena 24 punti di media poiché rimane bloccato in drilling laterale senza riuscire a progredire verso il basso.

I risultati della valutazione confermano le ipotesi iniziali con alcune sfumature importanti.

RL+LLM Reward emerge come l'approccio più efficace con win rate del 90% e score medio di 1 353 punti. L'approccio ibrido beneficia di entrambi i paradigmi: il ciclo RL fornisce reattività real-time, mentre il reward LLM introduce una pressione semantica verso comportamenti strategicamente corretti (scendere, raccogliere ossigeno, evitare blocchi X) senza richiedere la specifica manuale esaustiva della funzione di reward. Particolarmente significativa è la bassa varianza della profondità (± 7.2 righe contro ± 29.7 del DQN puro): il segnale LLM produce un comportamento più *robusto* e *consistente*.

Rainbow DQN con 536 400 step di training raggiunge il 70% di win rate. Il 20% di episodi stuck indica che l'agente cristallizza occasionalmente in loop di oscillazione non intercettati dal timeout. Con training esteso (GPU disponibile, completamento del curriculum) questi episodi stuck si ridurrebbero significativamente.

LLM Action fallisce completamente (0% win rate) per incompatibilità architettonica fondamentale: un modello da 8B parametri interrogato via API remota non può competere temporalmente con un ciclo di controllo real-time. Il fallimento non è imputabile alla qualità del ragionamento del modello (la consistenza del 100% dimostra che esso fornisce risposte coerenti per gli stessi stati) ma alla traduzione semantica della griglia simbolica: il modello interpreta il drilling laterale come mezzo per "trovare un percorso verso il basso", anziché eseguire direttamente DRILL_D.

7 Conclusioni

Questo progetto ha sviluppato e confrontato tre paradigmi distinti per il controllo autonomo di un agente nel videogioco *Mr. Driller*: un agente di Deep Reinforcement Learning basato su Rainbow DQN, un agente guidato direttamente da un Large Language Model per la selezione delle azioni, e un agente RL addestrato con una funzione di reward generata da LLM.

Il vincolo computazionale come fattore determinante

La sfida più pervasiva e trasversale dell'intero progetto non è stata tecnica nel senso stretto del termine, ma *infrastrutturale*: la totale assenza di una GPU dedicata ha imposto vincoli che hanno condizionato ogni scelta progettuale. L'addestramento dell'agente Rainbow DQN è stato condotto interamente su CPU commodity per 536 400 step, un numero considerevolmente inferiore a quanto riportato nella letteratura di riferimento su hardware GPU. Con una singola GPU di fascia media (ad esempio una NVIDIA RTX 3080) il curriculum di training potrebbe essere completato in poche ore, verificando se la cristallizzazione in ottimi locali osservata è un artefatto della scarsità computazionale o un limite intrinseco dell'architettura.

Il vincolo computazionale ha condizionato in modo differente anche i due componenti LLM. Per l'LLM Reward Model, la latenza di inferenza remota è gestibile perché il reward viene generato *offline* durante il training, non in tempo reale. Per l'LLM Action Generator, invece, la latenza è un vincolo architetturale che non può essere aggirato con ottimizzazioni di rete.

I limiti strutturali degli LLM come decisori diretti

L'impiego degli LLM come policy diretta per un task di controllo real-time ha evidenziato un limite di *compatibilità architettonica* tra due paradigmi computazionali fondamentalmente diversi. Un agente RL opera con latenza nell'ordine dei millisecondi su una rete neurale compatta specializzata; un LLM remoto opera con latenza nell'ordine dei secondi su un modello generalista. La riduzione della latenza da 17 secondi a 0.5–1.5 secondi ottenuta tramite cache LRU, streaming early exit e keepalive HTTP/2 è un risultato ingegneristicamente valido, ma non supera la soglia di reattività richiesta dal gioco.

Un secondo limite strutturale è il *dilemma prompt-latenza*: un prompt più ricco permette al modello di prendere decisioni migliori, ma aumenta il numero di token di input e quindi la latenza di inferenza. Non esiste un punto di equilibrio che soddisfi contemporaneamente i requisiti di qualità e velocità per il gioco in tempo reale.

Un terzo limite, meno ovvio, è la *non stazionarietà del contesto*: l'LLM risponde a ogni prompt in modo indipendente, senza memoria delle chiamate precedenti (a meno di non includere la storia nel prompt). In un gioco dove le conseguenze di un'azione si manifestano molti frame dopo, il modello non ha accesso al suo storico decisionale, rendendo impossibile l'apprendimento di policy temporalmente coerenti.

Il valore dell'approccio ibrido RL+LLM Reward Model

La variante ibrida che usa l'LLM come generatore della funzione di reward ha prodotto i risultati migliori (90% win rate), evitando completamente il problema della latenza real-time e sfruttando la conoscenza pregressa dell'LLM per definire implicitamente le

intenzioni del progettista senza una specifica manuale esaustiva. Il risultato più interessante è la bassa varianza del comportamento ($\sigma_{\text{depth}} = 7.2$ righe vs 29.7 del DQN puro): il segnale semantico LLM produce policy più robuste e generalizzabili.

Proposte per sviluppi futuri

Le direzioni di sviluppo più promettenti identificate sono tre. La prima è il completamento del curriculum di training su GPU dedicata. La seconda è la riduzione della latenza dell'LLM Action Generator attraverso modelli distillati e quantizzati eseguibili localmente (es. Llama 3.2-3B in formato GGUF via `llama.cpp`, latenza tipica 100–300 ms su CPU). La terza, concettualmente più ambiziosa, è la progettazione di architetture ibride in cui l'LLM non sostituisce la policy ma la *integra* come modulo di pianificazione a lungo termine: interrogato solo in situazioni di alta incertezza o a inizio episodio, mentre la policy RL gestisce le decisioni frame-by-frame. Questa separazione temporale delle responsabilità permetterebbe di sfruttare i punti di forza complementari dei due paradigmi senza ereditarne i rispettivi limiti.

Considerazioni finali

La lezione principale che emerge da questo progetto è che l'integrazione dei Large Language Models nei sistemi di Reinforcement Learning applicati al controllo real-time richiede un ripensamento architetturale profondo. I LLM non sono policy network più intelligenti: sono sistemi di ragionamento generalista con requisiti di latenza incompatibili con i loop di controllo ad alta frequenza. Usarli invece come componenti di pianificazione a lungo termine, come generatori di funzioni di reward, o come sistemi di annotazione offline delle transizioni, è una direzione più promettente che merita ulteriore esplorazione.

Dal punto di vista del Deep Reinforcement Learning, il progetto conferma che Rainbow DQN rimane un'architettura solida ed efficace per domini con fisica deterministica e reward densa. La combinazione di Double DQN, Dueling Networks, Noisy Nets, PER e N-step Returns ha dimostrato convergenza stabile e comportamenti emergenti sofisticati — gestione autonoma dell'ossigeno, schivata reattiva dei blocchi cadenti — che non erano stati esplicitamente programmati nella funzione di reward. Il principale ostacolo alla piena espressione di queste capacità rimane la disponibilità di risorse computazionali adeguate: un problema pratico e contingente, non un limite teorico dell'approccio.

Riferimenti bibliografici

[1] Gruber et al., *Mr. Driller Clone (open source)*, GitHub repository, 2023.

[2] M. Hessel, J. Modayil, H. van Hasselt et al., "Rainbow: Combining Improvements in Deep Reinforcement Learning," *Proc. AAAI Conference on Artificial Intelligence*, 2018.

[3] M. G. Bellemare, W. Dabney, R. Munos, "A Distributional Perspective on Reinforcement Learning," *Proc. ICML*, 2017.

[4] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[5] H. Larochelle, G. Hinton, "Learning to combine foveal glimpses with a third-order Boltzmann machine," *Proc. NeurIPS*, 2010.

[6] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *Proc. CVPR*, 2016.

[7] A. Vaswani et al., "Attention Is All You Need," *Proc. NeurIPS*, 2017.

[8] M. Fortunato et al., "Noisy Networks for Exploration," *Proc. ICLR*, 2018.

[9] K. He, X. Zhang, S. Ren, J. Sun, "Delving Deep into Rectifiers," *Proc. ICCV*, 2015.

[10] H. van Hasselt, A. Guez, D. Silver, "Deep Reinforcement Learning with Double Q-learning," *Proc. AAAI*, 2016.

[11] T. Schaul, J. Quan, I. Antonoglou, D. Silver, "Prioritized Experience Replay," *Proc. ICLR*, 2016.

[12] I. Loshchilov, F. Hutter, "Decoupled Weight Decay Regularization," *Proc. ICLR*, 2019.

[13] Z. Wang et al., "Dueling Network Architectures for Deep Reinforcement Learning," *Proc. ICML*, 2016.

[14] A. Y. Ng, D. Harada, S. Russell, "Policy Invariance Under Reward Transformations," *Proc. ICML*, 1999.

[15] Y. Bengio, J. Louradour, R. Collobert, J. Weston, "Curriculum Learning," *Proc. ICML*, 2009.